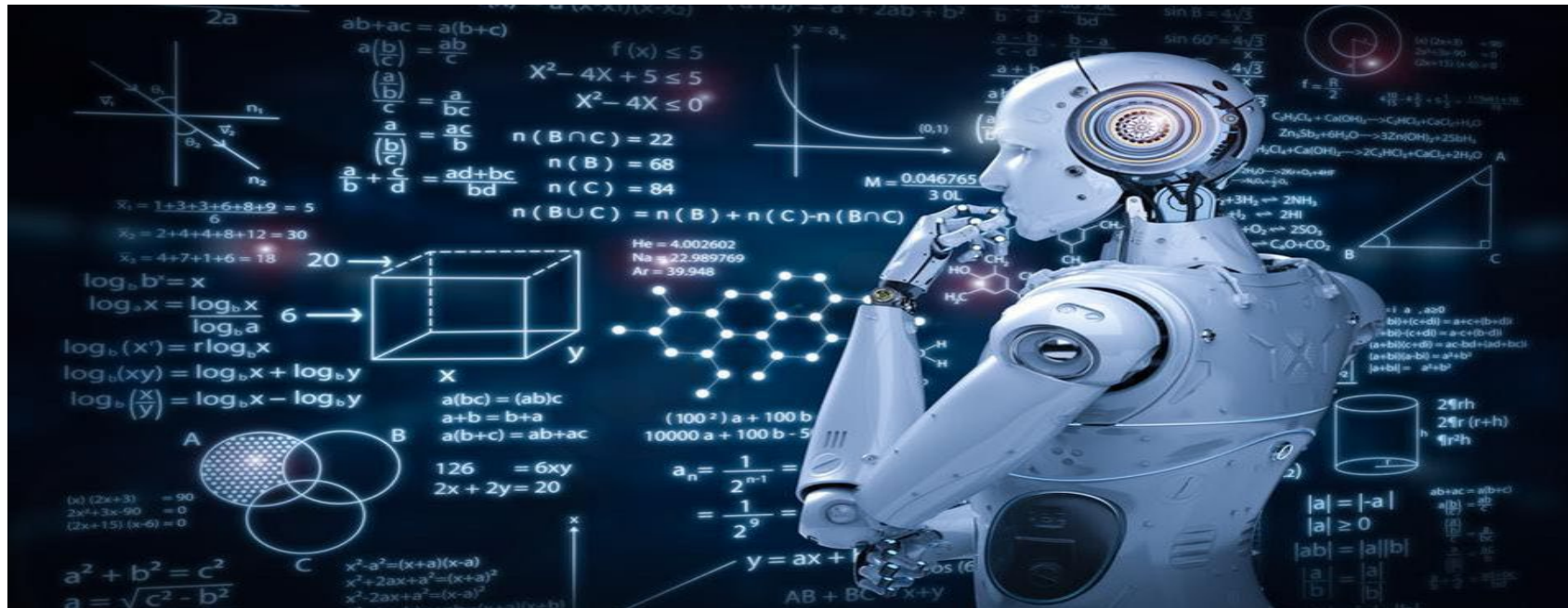


Lovely Professional University
Department of Computer Science & Engineering

Presentation on

INT254-Foundations of Machine Learning



Presented By:

Dr. Premananda Sahu

Department of Machine Learning

Topics to be covered...

- What is Machine Learning?
- History and Evolution of Machine Learning
- Types of Machine Learning
- Applications of Machine Learning
- Basic Concepts of Linear Algebra
- Vectors, Matrices, and Operations
- Probability Theory and Statistics
- Bayes' Theorem

What is Machine Learning?

- allows computers to learn and improve from data without being explicitly programmed.
- Uses algorithms to detect patterns within large datasets.
- The more data it processes, the more accurate the predictions become.
- Models trained to make intelligent predictions.
- It automates decision-making processes based on data analysis.
- used in various fields like healthcare, finance, and technology.
- Models improve over time by learning from errors and new data.

Key Components of Machine Learning

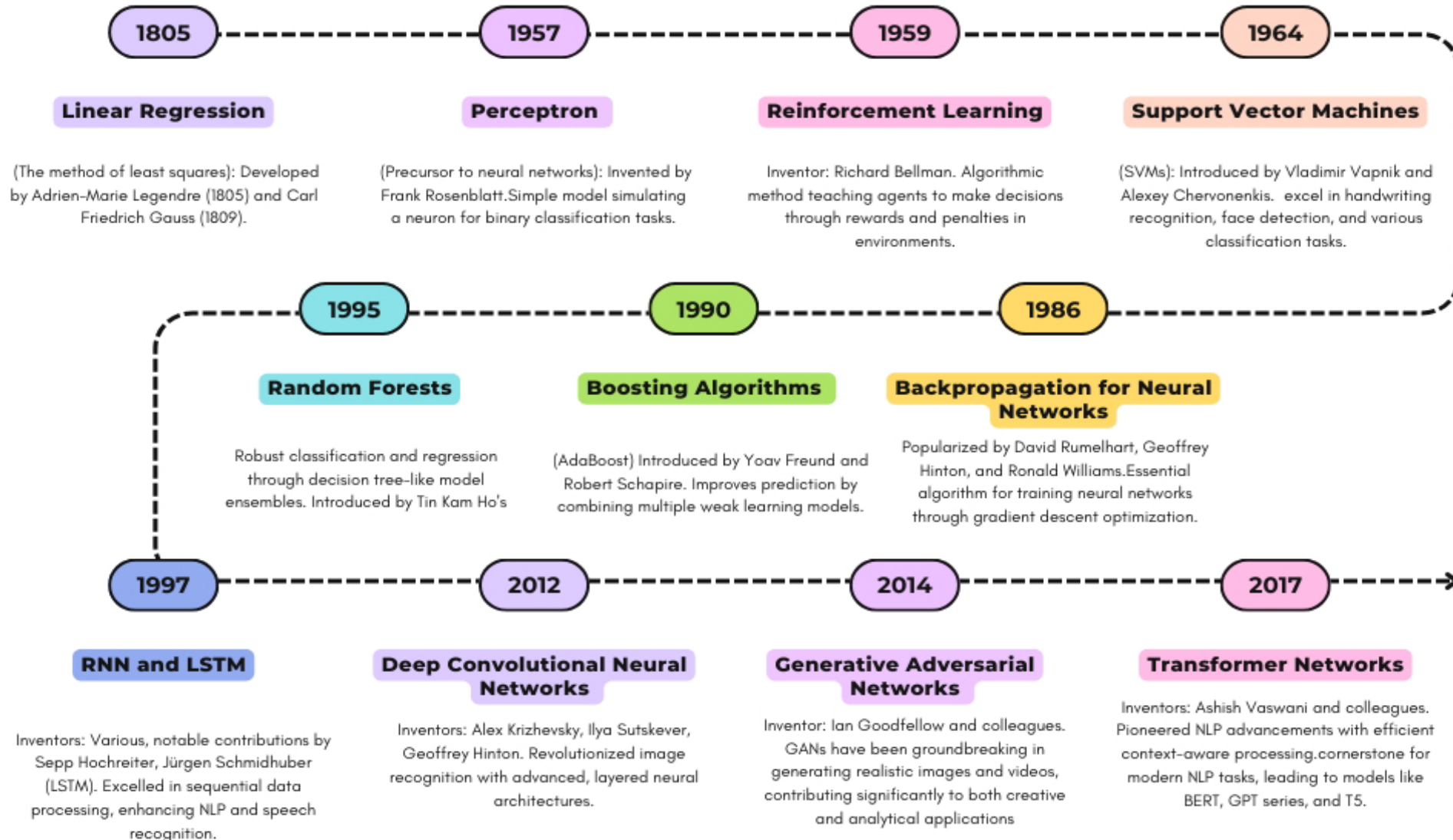
- Data Collection: collecting data that will be used to train the model.
- Data Preparation: This involves cleaning and structuring the data for analysis.
- Model Selection: Selecting the right algorithm that suits the problem at hand.
- Model Training: The chosen model is trained using the collected and prepared data.
- Model Evaluation: Evaluating the model's accuracy and performance using various metrics.
- Model Deployment: Deploying the trained model for real-world use in applications.

Example:

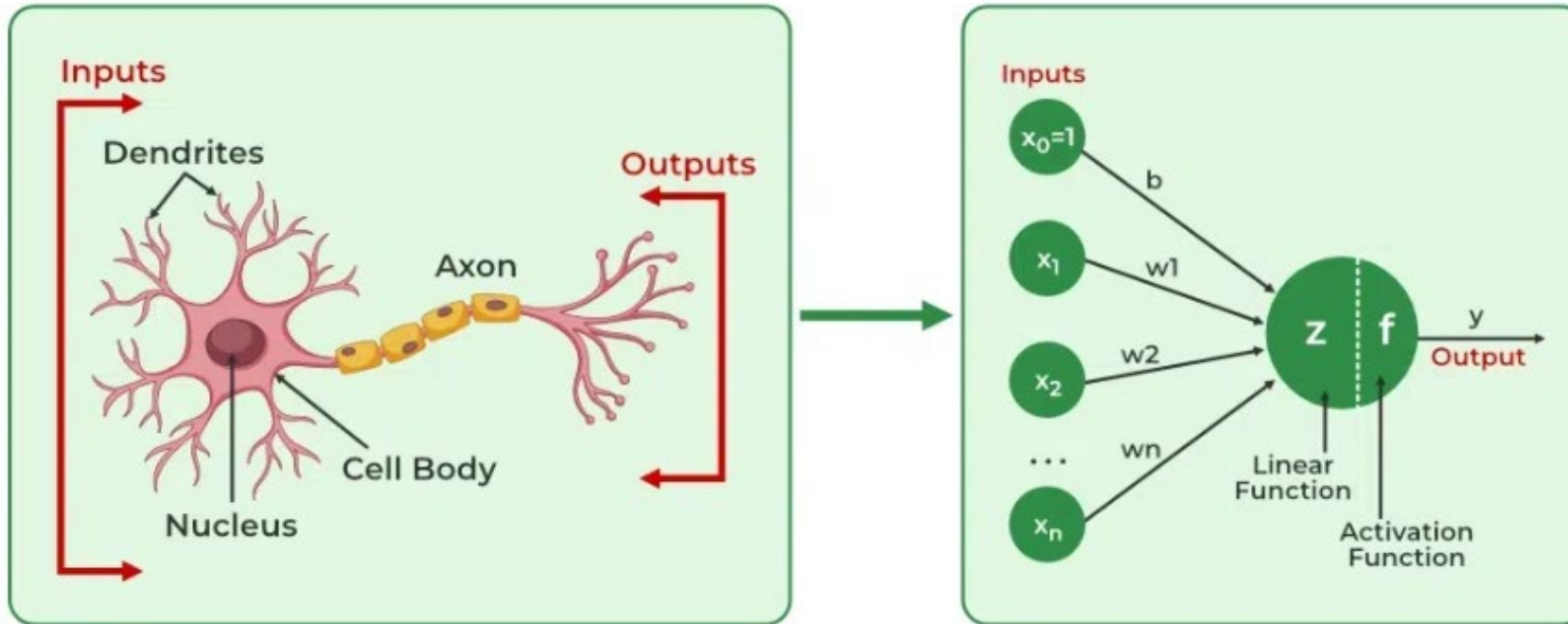
It is hard to say what makes a 2:



History and Evolution of Machine Learning



History and Evolution of Machine Learning

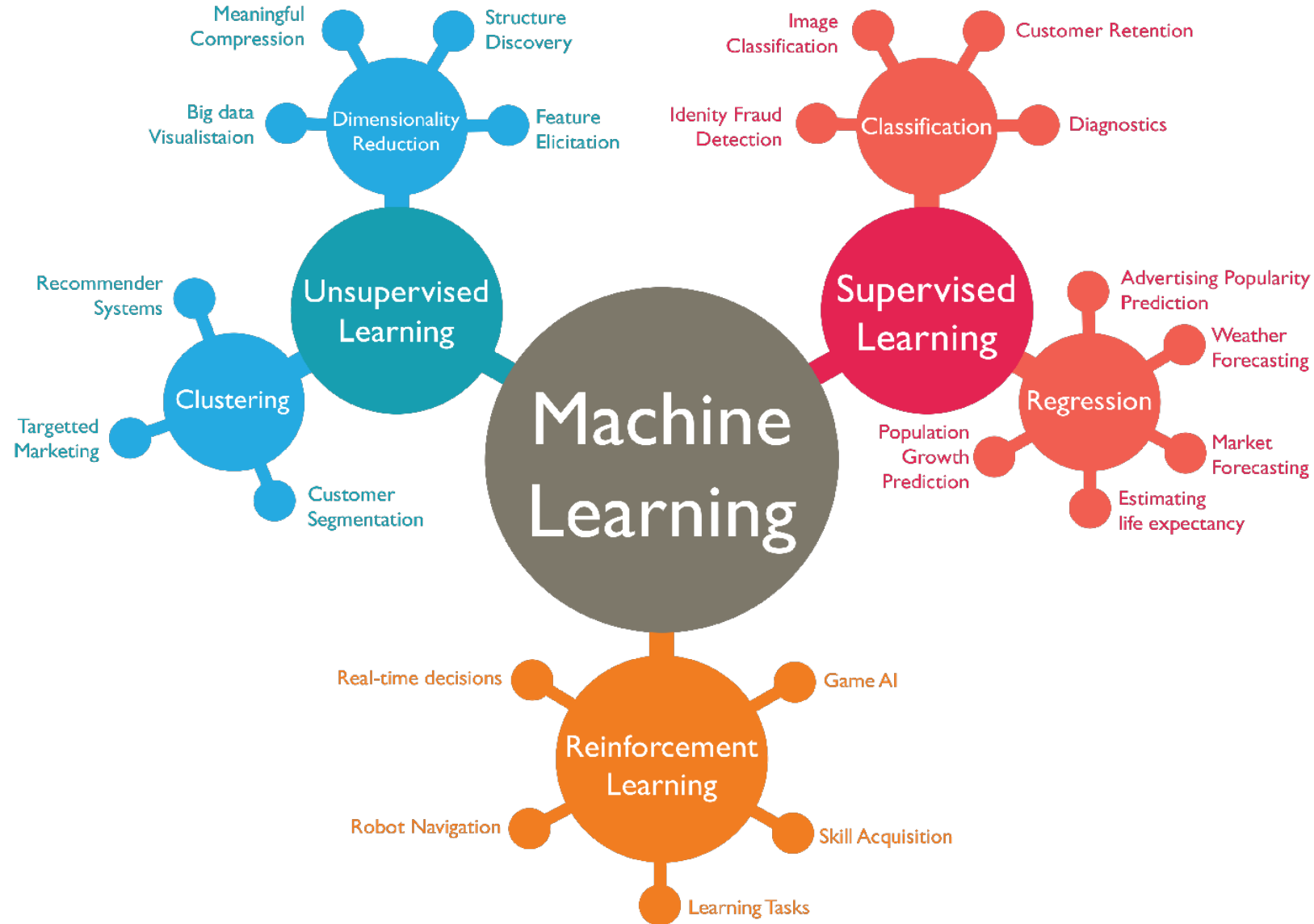


analogy between a biological neuron and an artificial neuron, showing how inputs are received and processed to produce outputs in both systems.

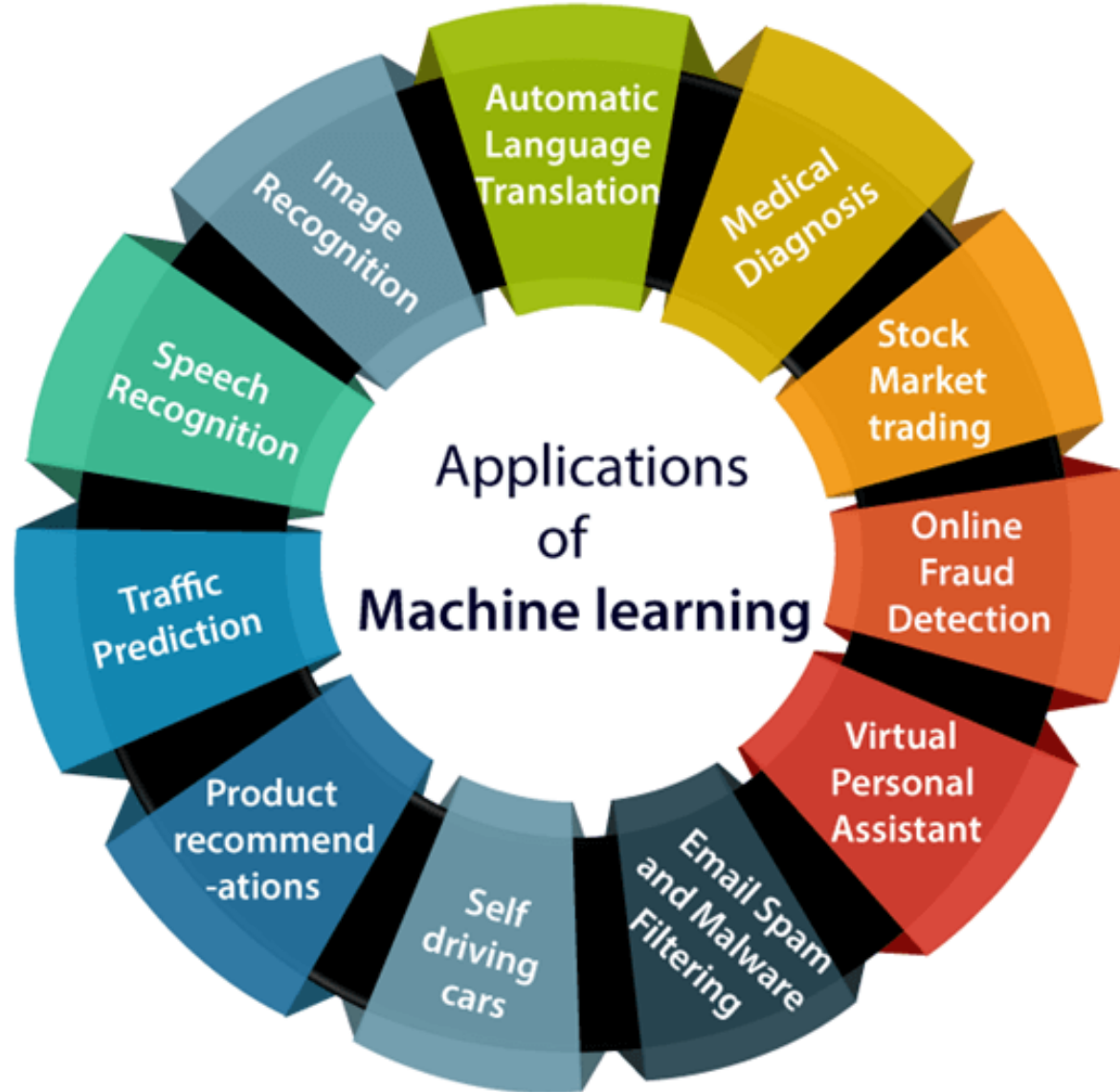
Evolution of Machine Learning

- Increased computational power and resources.
- Availability of large-scale datasets.
- Advancements in neural network architectures.
- Improved algorithms for model training.
- Integration with big data technologies.
- Applications in diverse industry sectors.

Types of Machine Learning



Applications of Machine Learning



Basic Concepts of Linear Algebra

- Describes vector which is a quantity that has both magnitude and direction.
- Represent data points as numerical arrays.
- Allowing algorithms to process and analyze information efficiently by organizing features into a structured format.
- enables operations like calculating distances and similarities to identify patterns and make predictions



Basic Concepts of Linear Algebra

- Used in mathematics, physics, engineering.
- Applications in machine learning and optimization.
- Essential tool for computational problem-solving.

Basic Concepts of Linear Algebra

Vectors

magnitude / size
AND
Direction

Displacement

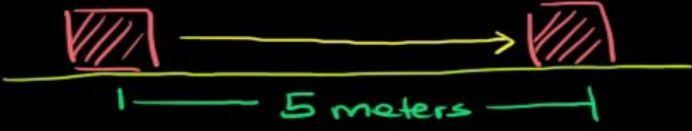
5 meters to the right

Vector

Scalars

magnitude / size

scalar
↓
5 meters
↖ distance



khanacademy.org

Vectors and Matrices

- A **scalar** is a number, like **3, -5, 0.368, etc,**
- A **vector** is a **list** of numbers (can be in a row or column),
- A **matrix** is an **array** of numbers (one or more rows, one or more columns).

Scalar

24

Vector

$\begin{bmatrix} 2 & -8 & 7 \end{bmatrix}$

row

or
column

$\begin{bmatrix} 2 \\ -8 \\ 7 \end{bmatrix}$

Matrix

$\begin{bmatrix} 6 & 4 & 24 \\ 1 & -9 & 8 \end{bmatrix}$

row(s) × column(s)

Vector Operations

Name	Vector Operation	Magnitude Model
Dot Product	$\mathbf{x} \cdot \mathbf{y}$	$ \mathbf{x} \mathbf{y} \cos(\theta_{xy})$
Outer Product	$\mathbf{xy}^T$	$ \mathbf{x} \mathbf{y} $
Addition	$\mathbf{x} + \mathbf{y}$	$\sqrt{ \mathbf{x} ^2 + \mathbf{y} ^2 + 2\mathbf{x} \cdot \mathbf{y}}$
Subtraction	$\mathbf{x} - \mathbf{y}$	$\sqrt{ \mathbf{x} ^2 + \mathbf{y} ^2 - 2\mathbf{x} \cdot \mathbf{y}}$
Matrix Multiplication	$A\mathbf{x}$	$\sigma \mathbf{x} $ $ \sigma_A ^{min} \leq \sigma \leq \sigma_A ^{max}$
Pointwise Product	$\mathbf{x} \circ \mathbf{y}$	$\epsilon \mathbf{x} \mathbf{y} $ $0 \leq \epsilon \leq 1$

Matrices Operations

Matrix Addition

$$\begin{bmatrix} 5 & 4 \\ 8 & 6 \\ 1 & 3 \end{bmatrix} + \begin{bmatrix} 9 & 2 \\ 8 & 6 \\ 3 & 0 \end{bmatrix} = \begin{bmatrix} 5+9 & 4+2 \\ 8+8 & 6+6 \\ 1+3 & 3+0 \end{bmatrix} = \begin{bmatrix} 14 & 6 \\ 16 & 12 \\ 4 & 3 \end{bmatrix}$$

Matrix Subtraction

$$\begin{bmatrix} 7 & 6 & 9 \\ 4 & 8 & 2 \end{bmatrix} - \begin{bmatrix} 6 & 9 & 1 \\ 2 & 5 & 7 \end{bmatrix} = \begin{bmatrix} 7-6 & 6-9 & 9-1 \\ 4-2 & 8-5 & 2-7 \end{bmatrix} = \begin{bmatrix} 1 & -3 & 8 \\ 2 & 3 & -5 \end{bmatrix}$$

Matrix Multiplication

$$\begin{bmatrix} 7 & 6 & 1 \\ 2 & 3 & 8 \end{bmatrix} * \begin{bmatrix} 4 & 9 \\ 1 & 7 \\ 3 & 8 \end{bmatrix} = \begin{bmatrix} 7*4+6*1+1*3 & 7*9+6*7+1*8 \\ 2*4+3*1+8*3 & 2*9+3*7+8*8 \end{bmatrix} = \begin{bmatrix} 37 & 113 \\ 35 & 103 \end{bmatrix}$$

Transpose Of A Matrix

$$\begin{bmatrix} 7 & 2 & 9 \\ 1 & 6 & 3 \end{bmatrix} \rightarrow \begin{bmatrix} 7 & 1 \\ 2 & 6 \\ 9 & 3 \end{bmatrix}$$

Probability Theory

- Probability implies 'likelihood' or 'chance'.
- The likelihood of rain tomorrow is high.
- There's a good chance we'll win the game today
- Hence the value of probability ranges from 0 to 1.

Probability Formula

E : Event

$$P(E) = \frac{\text{Number of favourable outcomes}}{\text{Total number of possible outcomes}}$$

$$0 \leq P(E) \leq 1$$

Conditional Probability

- Possibility of an event depends upon the existence of a previous event.
- likelihood of an outcome occurring based on a previous outcome in similar circumstances.
- Example: If it rains, what's the probability you'll carry an umbrella?

Conditional Probability

$$P(B|A) = \frac{P(B \cap A)}{P(A)}$$

Conditional Probability

- We flip two coins. What is the probability that both are heads, given that at least one of them is heads?
 - We are given that there is at least one head.. So there are three possibilities HH, HT, TH. Only one outcome has both coins as heads: HH.
 - So our answer is = $1/3$
-
- We roll a six-sided die. What is the probability that the roll is a 6, given that the outcome is an even number
 - The possible outcomes when rolling a die are: 1, 2, 3, 4, 5, 6.
 - The even numbers are: 2, 4, 6 (3 outcomes).
 - Only one of these is a 6.
 - So our answer = $1/3$

Statistics

Mean, Median, Mode and Range

Mean

Add all the numbers then divide by the amount of numbers

9, 3, 1, 8, 3, 6

$$9 + 3 + 1 + 8 + 3 + 6 = 30$$

$$30 \div 6 = 5$$

The mean is 5

Median

Order the set of numbers, the median is the middle number

9, 3, 1, 8, 3, 6

1, 3, 3, 6, 8, 9

The median is 4.5

Mode

The most common number

9, 3, 1, 8, 3, 6

The mode is 3

Range

The difference between the highest number and lowest number

9, 3, 1, 8, 3, 6

$$9 - 1 = 8$$

The range is 8

Bayes' Theorem

- Mathematical formula for conditional probability
- providing a framework to update the probability of a hypothesis based on new evidence.
- Named Based on prior knowledge and evidence
- allowing algorithms to make more accurate predictions
- after **Thomas Bayes, statistician**
- Used in various fields today
- Foundation of Bayesian statistics

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

$$P(B|A) = \frac{P(B \cap A)}{P(A)}$$

Bayes' Theorem

Bayes' Theorem

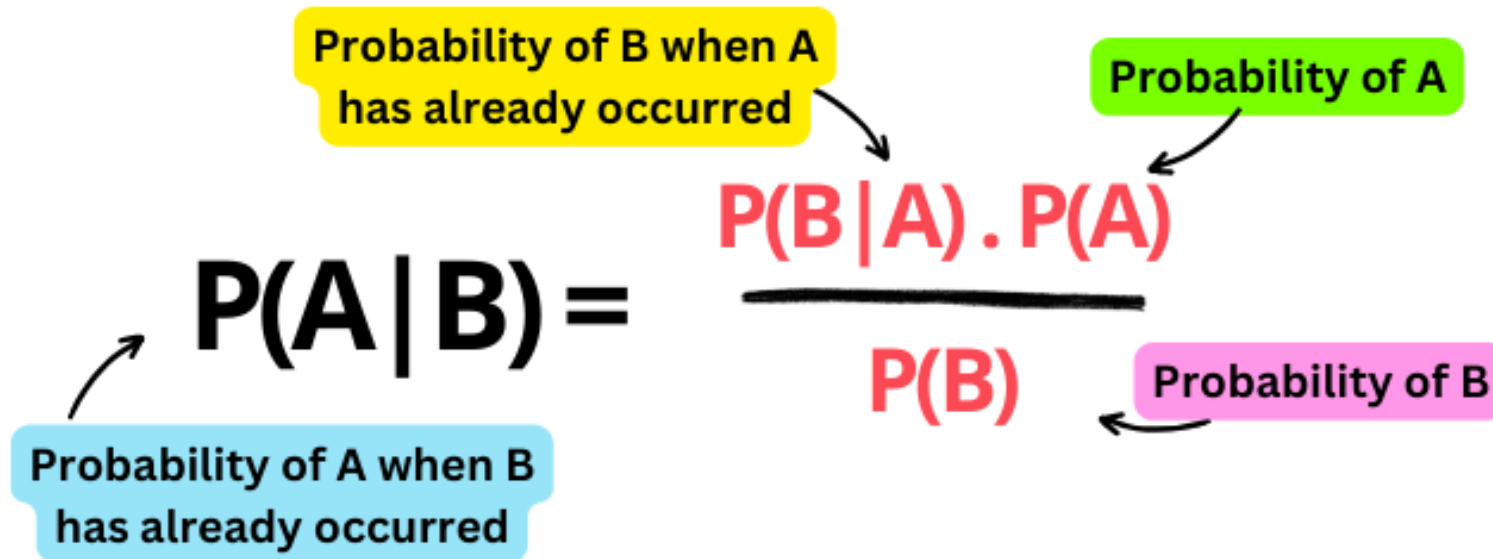


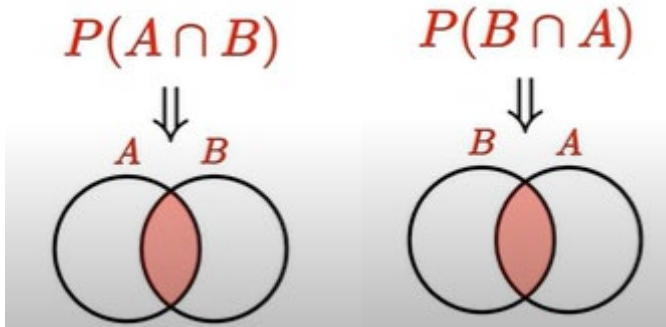
Diagram illustrating Bayes' Theorem:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Labels and their corresponding parts in the formula:

- Probability of A when B has already occurred** (light blue box) points to $P(A|B)$.
- Probability of B when A has already occurred** (yellow box) points to $P(B|A)$.
- Probability of A** (green box) points to $P(A)$.
- Probability of B** (pink box) points to $P(B)$.

Bayes' Theorem



$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

$$P(A) \cdot P(B|A) = \frac{P(A \cap B)}{P(A)} \cdot P(A)$$

$$P(A) \cdot P(B|A) = \frac{P(A \cap B)}{\cancel{P(A)}} \cdot \cancel{P(A)}$$

$$P(B|A) \cdot P(A) = P(A \cap B)$$

Bayes' Theorem

Ex You've been planning a picnic for your family. You're trying to decide whether to postpone due to rain. The chance of rain on any day is 15%. The morning of the picnic, it's cloudy. The prob. of it being cloudy is 25% and on days where it rains, it's cloudy in the morning 80% of the time.

Should you postpone the picnic?

$$P(\text{rain}) = 0.15$$

$$P(\text{cloudy}) = 0.25$$

$$P(\text{cloudy}|\text{rain}) = 0.80 \quad P(\text{rain}|\text{cloudy}) = \frac{P(\text{cloudy}|\text{rain}) \cdot P(\text{rain})}{P(\text{cloudy})}$$

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Answer

Given Data:

- $P(\text{rain}) = 0.15$ (Prior probability of rain)
- $P(\text{cloudy}) = 0.25$ (Total probability of cloudy)
- $P(\text{cloudy}|\text{rain}) = 0.80$ (Likelihood of cloudy given rain)

We need to compute:

$$P(\text{rain}|\text{cloudy}) = \frac{P(\text{cloudy}|\text{rain}) \cdot P(\text{rain})}{P(\text{cloudy})}$$

Step-by-Step Calculation:

$$\begin{aligned} P(\text{rain}|\text{cloudy}) &= \frac{0.80 \times 0.15}{0.25} \\ &= \frac{0.12}{0.25} \\ &= 0.48 \end{aligned}$$

Final Answer:

The probability that it will rain given that it is cloudy is **0.48 (or 48%)**.

Since there's a **48% chance of rain**, you might consider postponing the picnic if that probability is too high for your plans.

Questions

- **Conditional Probability:**
- In a bag of 10 marbles (4 red, 3 blue, 3 green), what is the probability of drawing a blue marble given that a red marble has already been drawn?

Since a red marble has already been drawn, we are left with 9 marbles in total. The number of blue marbles remains the same (3).

The probability of drawing a blue marble after a red one has already been drawn is:

$$P(\text{Blue}|\text{Red drawn}) = \frac{\text{Remaining blue marbles}}{\text{Remaining total marbles}} = \frac{3}{9} = \frac{1}{3}$$

So, the probability is $\frac{1}{3}$ or approximately 0.333 (33.3%).

- What is the probability of rolling a 3 on a fair six-sided die?

Questions

- **Basic Probability:**
- If you draw one card from a standard deck of 52 cards, what is the probability of drawing an Ace?
- In a classroom of 20 students (12 boys and 8 girls), what is the probability of selecting a girl given that a boy was selected first without replacement?

Questions

- **Bayes theorem:**
- A factory produces 95% non-defective items and 5% defective items. A quality control test correctly identifies a defective item 99% of the time and a non-defective item 98% of the time. What is the probability that an item is defective given that it has failed the quality control test?

Questions

- **Bayes theorem:**
- A factory produces 95% non-defective items and 5% defective items. A quality control test correctly identifies a defective item 99% of the time and a non-defective item 98% of the time. What is the probability that an item is defective given that it has failed the quality control test?

We need to find the probability that an item is defective given that it has failed the quality control test. This is a case of Bayes' Theorem, which states:

$$P(D|F) = \frac{P(F|D)P(D)}{P(F)}$$

where:

- $P(D) = 0.05$ (probability of a defective item)
- $P(N) = 0.95$ (probability of a non-defective item)
- $P(F|D) = 0.99$ (probability that a defective item fails the test)
- $P(F|N) = 1 - 0.98 = 0.02$ (probability that a non-defective item fails the test)

Step 1: Compute Total Probability of Failure ($P(F)$)

Using the Law of Total Probability:

$$P(F) = P(F|D)P(D) + P(F|N)P(N)$$

$$P(F) = (0.99 \times 0.05) + (0.02 \times 0.95)$$

$$P(F) = 0.0495 + 0.019 = 0.0685$$

Step 2: Apply Bayes' Theorem

$$P(D|F) = \frac{0.99 \times 0.05}{0.0685}$$

$$P(D|F) = \frac{0.0495}{0.0685} \approx 0.723$$

Final Answer:

The probability that an item is defective given that it has failed the quality control test is 0.723 or 72.3%.

Lovely Professional University
Department of Computer Science & Engineering
Presentation on

INT254-Foundations of Machine Learning



Presented By:
Dr. Premananda Sahu
Department of Machine Learning

Topics to be covered...

- Data handling with Numpy and Pandas: Types of data
- Creating arrays with Numpy
- Array operations
- Linear algebra operations using Numpy
- Reading the data using Pandas
- Handling missing data
- Data cleaning and preparation

Types of Data

- **Numerical Data:** Continuous or discrete data (e.g., sales, age).
- **Categorical Data:** Data that can be divided into categories (e.g., gender, color).
- **Text Data:** Strings or textual information (e.g., names, comments).
- **Time-Series Data:** collected or recorded at specific time intervals. (e.g., stock prices over days).
- **Boolean Data:** Binary data (True/False values).

NumPy

- Python library used for working with arrays.
- The array object in NumPy is called ndarray (n-dimensional).
- Create a NumPy ndarray object by using the array() function.
- **Example-:**
- `import numpy as np`
- `arr = np.array([1, 2, 3, 4, 5])`
- `print(arr)`
- `print(type(arr))`

Basic Array Operations

- Traverse – print all the array elements one by one.
- Insertion – Adds an element at the given index.
- Deletion – Deletes an element at the given index.
- Search – Searches an element using the given index or by the value.
- Update – Updates an element at the given index.

Traverse Operations

- from array import * *//* is you want to import all functions from the array module//*
- array1 = array('i', [10,20,30,40,50])
- for x in array1:
- print(x)
- Output-:
- 10
- 20
- 30
- 40
- 50

Insert Operations

- insert one or more data elements into an array.
- a new element can be added at the beginning, end, or any given index of array.
- We add a data element at the middle of the array using the python in-built insert() method.
- **Example-:**
- `from array import *`
- `array1 = array('i', [10,20,30,40,50])`
- `array1.insert(1,60)`
- `for x in array1:`
- `print(x)`
- Output-: 10
- 60
- 20
- 30
- 40
- 50

Deletion Operations

- Refers to removing an existing element from the array
- **Example-:**
- `from array import *`
- `array1 = array('i', [10,20,30,40,50])`
- `array1.remove(40)`
- `for x in array1:`
- `print(x)`
- **Output-:**
- 10
- 20
- 30
- 50

Search & Update Operations

- Refers to removing an existing element from the array
 - **Example-:**
 - from array import *
 - array1 = array('i', [10,20,30,40,50])
 - print (array1.index(40))
 - Output-:
 - 3
- from array import *
 - array1 = array('i', [10,20,30,40,50])
 - array1[2] = 80
 - for x in array1:
 - print(x)
 - Output-:
 - 10
 - 20
 - 80
 - 40
 - 50

Array Operations

- **Element-wise Operations:** Addition, subtraction, multiplication, division (e.g., `array1 + array2`).
- **Aggregation Functions:** `np.sum()`, `np.mean()`, `np.min()`, `np.max()`.
- **Indexing and Slicing:** Access specific elements or subarrays (e.g., `array[1, 2]`, `array[:, 1]`).
- **Broadcasting:** Operations between arrays of different shapes.

Array Operations

- **Broadcasting:** Operations between arrays of different shapes.
- numpy treats arrays with different Dimension during arithmetic operations.
- Example-:
- import numpy as np
- `a = np.array([5, 7, 3, 1])`
- `b = np.array([90, 50, 0, 30])`
- # array are compatible because of same Dimension
- `c = a * b`
- `print(c)`
- Output-: `[450 350 0 30]`

Linear Algebra Operations using NumPy

- **Dot Product:** `np.dot(array1, array2)` (sum of the element-wise product of their components.)
- `import numpy as np`
- `a = np.array([1, 2, 3])`
- `b = np.array([4, 5, 6])`
- `dot_product = np.dot(a, b)`
- `print(dot_product)` # Output: $1*4 + 2*5 + 3*6 = 32$

Linear Algebra Operations using NumPy

- **Matrix Multiplication:** `np.matmul(array1, array2)` or `array1 @ array2`
- `A = np.array([[1, 2], [3, 4]])`
- `B = np.array([[5, 6], [7, 8]])`
- `matmul_result = np.matmul(A, B)`
- `print(matmul_result)`
- # Output:
- # `[[19 22]`
- # `[43 50]]`
- # Alternative syntax using @ operator
- `matmul_result2 = A @ B`
- `print(matmul_result2) # Same output`

Linear Algebra Operations using NumPy

- **Transpose:** `np.transpose(array)`

```
A = np.array([[1, 2, 3], [4, 5, 6]])
```

```
transpose_A = np.transpose(A)
```

```
print(transpose_A)
```

Output:

```
# [[1 4]
```

```
# [2 5]
```

```
# [3 6]]
```

Linear Algebra Operations using NumPy

- **Inverse:** `np.linalg.inv(matrix)`

```
import numpy as np
```

```
# Define a 2x2 matrix
```

```
A = np.array([[4, 7], [2, 6]])
```

```
# Compute the inverse using np.linalg.inv()
```

```
A_inv = np.linalg.inv(A)
```

```
# Print the inverse matrix
```

```
print("Inverse of A:\n", A_inv)
```

```
# Verify: Multiplying A with A_inv should give the identity matrix
```

```
identity_matrix = np.round(np.dot(A, A_inv))
```

```
print("\nIdentity Matrix (A * A_inv):\n", identity_matrix)
```

$$A = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}$$

Step 1: Compute the Determinant

$$\det(A) = (4 \times 6) - (7 \times 2) = 24 - 14 = 10$$

Since $\det(A) \neq 0$, the matrix is invertible.

Step 2: Compute the Cofactor Matrix

For a 2×2 matrix, the cofactor matrix is:

$$\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Substituting values:

$$\begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix}$$

Step 3: Compute the Adjugate (Transpose of Cofactor Matrix)

Since it's a 2×2 matrix, the cofactor matrix is already symmetric, so its transpose is the same:

$$\text{Adj}(A) = \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix}$$

Step 4: Compute the Inverse

$$A^{-1} = \frac{1}{\det(A)} \cdot \text{Adj}(A)$$

$$A^{-1} = \frac{1}{10} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix}$$

$$A^{-1} = \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix}$$

Verification

Multiplying A with A^{-1} should give the identity matrix:

$$\begin{aligned} A \cdot A^{-1} &= \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix} \times \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix} \\ &= \begin{bmatrix} (4 \times 0.6) + (7 \times -0.2) & (4 \times -0.7) + (7 \times 0.4) \\ (2 \times 0.6) + (6 \times -0.2) & (2 \times -0.7) + (6 \times 0.4) \end{bmatrix} \\ &= \begin{bmatrix} 2.4 - 1.4 & -2.8 + 2.8 \\ 1.2 - 1.2 & -1.4 + 2.4 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \end{aligned}$$

This confirms that A^{-1} is correct!

Linear Algebra Operations using NumPy

- **Eigenvalues/Eigenvectors:** `np.linalg.eig(matrix)`
- `A = np.array([[4, -2], [1, 1]])`
- `eigenvalues, eigenvectors = np.linalg.eig(A)`
- `print("Eigenvalues:", eigenvalues)`
- `print("Eigenvectors:\n", eigenvectors)`
- # Output:
- # Eigenvalues: [3. 2.]
- # Eigenvectors:
- # `[[ 0.89442719 0.70710678]`
- # `[ 0.4472136 0.70710678]]`

Mathematical Method to Find Eigenvalues and Eigenvectors

For a given square matrix A , the eigenvalues and eigenvectors satisfy the equation:

$$Av = \lambda v$$

where:

- λ is an eigenvalue (a scalar).
 - v is the corresponding eigenvector (a nonzero vector).
-

Steps to Find Eigenvalues and Eigenvectors

Step 1: Compute the Characteristic Equation

The characteristic equation is obtained from:

$$\det(A - \lambda I) = 0$$

where:

- I is the identity matrix.
- $A - \lambda I$ is the matrix obtained by subtracting λ from the diagonal elements of A .
- $\det(\cdot)$ denotes the determinant.

Solving this determinant equation gives the eigenvalues λ .

Step 2: Solve for Eigenvectors

For each eigenvalue λ , solve the equation:

$$(A - \lambda I)v = 0$$

This is a system of linear equations, where the solutions give the eigenvectors.

Linear



$$A = \begin{bmatrix} 4 & -2 \\ 1 & 1 \end{bmatrix}$$

Step 1: Compute the Characteristic Equation

First, subtract λ from the diagonal elements:

$$A - \lambda I = \begin{bmatrix} 4 - \lambda & -2 \\ 1 & 1 - \lambda \end{bmatrix}$$

Find the determinant:

$$\begin{aligned} \det(A - \lambda I) &= (4 - \lambda)(1 - \lambda) - (-2)(1) \\ &= (4 - \lambda)(1 - \lambda) + 2 \\ &= 4 - 4\lambda - \lambda + \lambda^2 + 2 \\ &= \lambda^2 - 5\lambda + 6 \end{aligned}$$

Set the determinant to zero:

$$\lambda^2 - 5\lambda + 6 = 0$$

Solve the quadratic equation:

$$(\lambda - 3)(\lambda - 2) = 0$$

So, the eigenvalues are:

$$\lambda_1 = 3, \quad \lambda_2 = 2$$

Step 2: Solve for Eigenvectors

For $\lambda_1 = 3$:

Substituting $\lambda = 3$ into $A - \lambda I$:

$$A - 3I = \begin{bmatrix} 4 - 3 & -2 \\ 1 & 1 - 3 \end{bmatrix} = \begin{bmatrix} 1 & -2 \\ 1 & -2 \end{bmatrix}$$

Solve:

$$\begin{bmatrix} 1 & -2 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This gives the equation:

$$x - 2y = 0 \quad \Rightarrow \quad x = 2y$$

Thus, the eigenvector corresponding to $\lambda_1 = 3$ is:

$$v_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

(any scalar multiple of this vector is also a valid eigenvector).

Linear Algebra Operations using NumPy

For $\lambda_2 = 2$:

Substituting $\lambda = 2$:

$$A - 2I = \begin{bmatrix} 4-2 & -2 \\ 1 & 1-2 \end{bmatrix} = \begin{bmatrix} 2 & -2 \\ 1 & -1 \end{bmatrix}$$

Solve:

$$\begin{bmatrix} 2 & -2 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This gives:

$$2x - 2y = 0 \Rightarrow x = y$$

Thus, the **eigenvector** corresponding to $\lambda_2 = 2$ is:

$$v_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Final Answer

The **eigenvalues** of A are:

$$\lambda_1 = 3, \quad \lambda_2 = 2$$

The corresponding **eigenvectors** are:

$$v_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Pandas

- Mostly used for data analysis tasks in Python.
- Functions for analyzing, cleaning, exploring, and manipulating data.
- Analyze big data and make conclusions based on statistical theories.
- Clean messy data sets, and make them readable and relevant.
- A DataFrame in Pandas is a 2D tabular structure.
- Similar to an Excel spreadsheet or SQL table.

Creating Pandas

- Example-:
- import pandas as pd

```
mydataset = {  
    'cars': ["BMW", "Volvo", "Ford"], 'passings': [3, 7, 2]  
}
```

```
myvar = pd.DataFrame(mydataset)
```

```
print(myvar)
```

- Output-:
- cars passings
- 0 BMW 3
- 1 Volvo 7
- 2 Ford 2

Pandas Series

- A Pandas Series is like a column in a table.
- Example-::
- Create a simple Pandas Series from a list:
- `import pandas as pd`

```
a = [1, 7, 2]
myvar = pd.Series(a)
print(myvar)
```

- Output-:
- 0 1
- 1 7
- 2 2
- dtype: int64(The type int64 tells us that pandas is storing each value within this column as a 64-bit integer)

Key/Value Objects as Series

- Example
- Create a simple Pandas Series from a dictionary:
- import pandas as pd

```
calories = {"day1": 420, "day2": 380, "day3": 390}
```

```
myvar = pd.Series(calories)
```

```
print(myvar)
```

- Output-:
- day1 420
- day2 380
- day3 390
- dtype: int64

Key/Value Objects as Series

- Example
- Create a Series using only data from "day1" and "day2":
- import pandas as pd

```
calories = {"day1": 420, "day2": 380, "day3": 390}
```

```
myvar = pd.Series(calories, index = ["day1", "day2"])
```

```
print(myvar)
```

- Output-:
- day1 420
- day2 380
- dtype: int64

Reading the Data Using Pandas

- Loading Data:

- CSV: `pd.read_csv('file.csv')`
- Excel: `pd.read_excel('file.xlsx')`
- JSON: `pd.read_json('file.json')` (data sent from server to web page)
- SQL: `pd.read_sql('query', connection)`

- Exploring Data:

- `df.head()`, `df.info()`, `df.describe()`, `df.columns`

- Exploring Data:

- `df.head()`, `df.info()`, `df.describe()`, `df.columns`

`import pandas as pd`

- `# Creating a sample DataFrame`

- `data = {'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],`

- `'Age': [25, 30, 35, 40, 45],`

- `'Salary': [50000, 60000, 70000, 80000, 90000]}`

- `df = pd.DataFrame(data)`

- `# Example usage`

- `print(df.head())` `# Displays the first 5 rows`

- `print(df.info())` `# Displays information about the DataFrame`

- `print(df.describe())` `# Displays statistical summary of numerical columns`

- `print(df.columns)` `# Displays column names`

MISSING VALUES IN PANDAS DATAFRAME

- Pandas provides two useful functions: `isnull()` and `notnull()`.
- These functions help detect whether a value is NaN or not.
- Making it easier to clean and preprocess data.
- `isnull()` used to locate and address missing data within a dataset.
- Returns a DataFrame of Boolean values.
- **True** represents missing data (NaN)
- **`notnull()`** indicates non-missing data.

- # Importing pandas and numpy
- import pandas as pd
- import numpy as np

- # Sample DataFrame with missing values
- data = {'First Score': [100, 90, np.nan, 95],
- 'Second Score': [30, 45, 56, np.nan],
- 'Third Score': [np.nan, 40, 80, 98]}

- df = pd.DataFrame(data)

- # Checking for missing values using isnull()
- missing_values = df.isnull()

- print(missing_values)

- # Importing pandas and numpy
- import pandas as pd
- import numpy as np

- # Sample DataFrame with missing values
- data = {'First Score': [100, 90, np.nan, 95],
 - 'Second Score': [30, 45, 56, np.nan],
 - 'Third Score': [np.nan, 40, 80, 98]}

- df = pd.DataFrame(data)

- # Checking for non-missing values using notnull()
- non_missing_values = df.notnull()

- print(non_missing_values)

Handling Missing Data

- **Identify Missing Data:** `df.isnull().sum()`
- **Fill Missing Values:**
 - `df.fillna(value)`
 - `df.fillna(method='ffill')`
- **Drop Missing Values:** `df.dropna()`
- **Replace Missing Values:** `df.replace(to_replace=np.nan, value=0)`

Data Cleaning and Preparation

- **Removing Duplicates:** `df.drop_duplicates()`
- **Renaming Columns:** `df.rename(columns={'old_name': 'new_name'})`
- **Changing Data Types:** `df['column'] = df['column'].astype('type')`

Coding Demo on Jupyter Notebook

Data Cleaning and Preparation

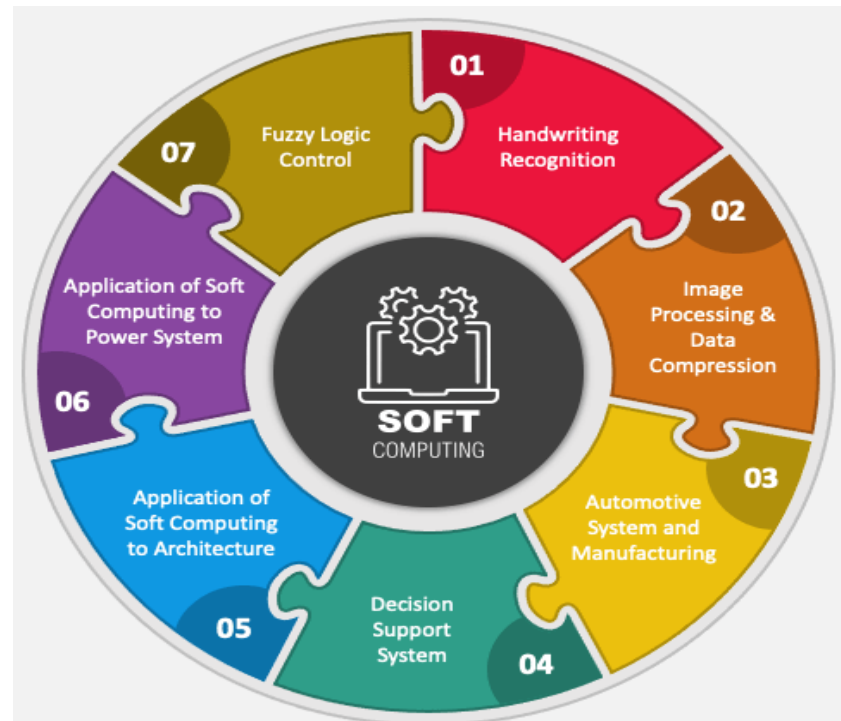
- **Filtering Data:** `df[df['column'] > value]`
- **Feature Scaling:** Normalize or standardize data (e.g., Min-Max Scaling)
- **Encoding Categorical Data:** Convert categories into numerical values (e.g., One-Hot Encoding with `pd.get_dummies()`)

Coding Demo on Jupyter Notebook

Lovely Professional University
Department of Computer Science & Engineering

Presentation on

INT254-Foundations of Machine Learning



Presented By:

Dr. Premananda Sahu

Department of Machine Learning

Topics to be covered...

- Soft Computing: What is Soft Computing?
- Comparison with Hard Computing
- Components of Soft Computing
- Neural Network: Biological Motivation and History
- Applications of Neural Networks
- Structure and Working of Artificial Neurons
- Perceptron Learning Algorithm
- Architecture and Working of Feedforward Neural Networks
- Activation functions
- Case study: Practical Problem-solving using Neural Networks

What is Soft Computing

- Fusion of **Fuzzy Logic, Neuro Computing & Evolutionary Computing**
- Designed to enable **solutions to real world problems**
- Solutions **which are difficult to model mathematically**

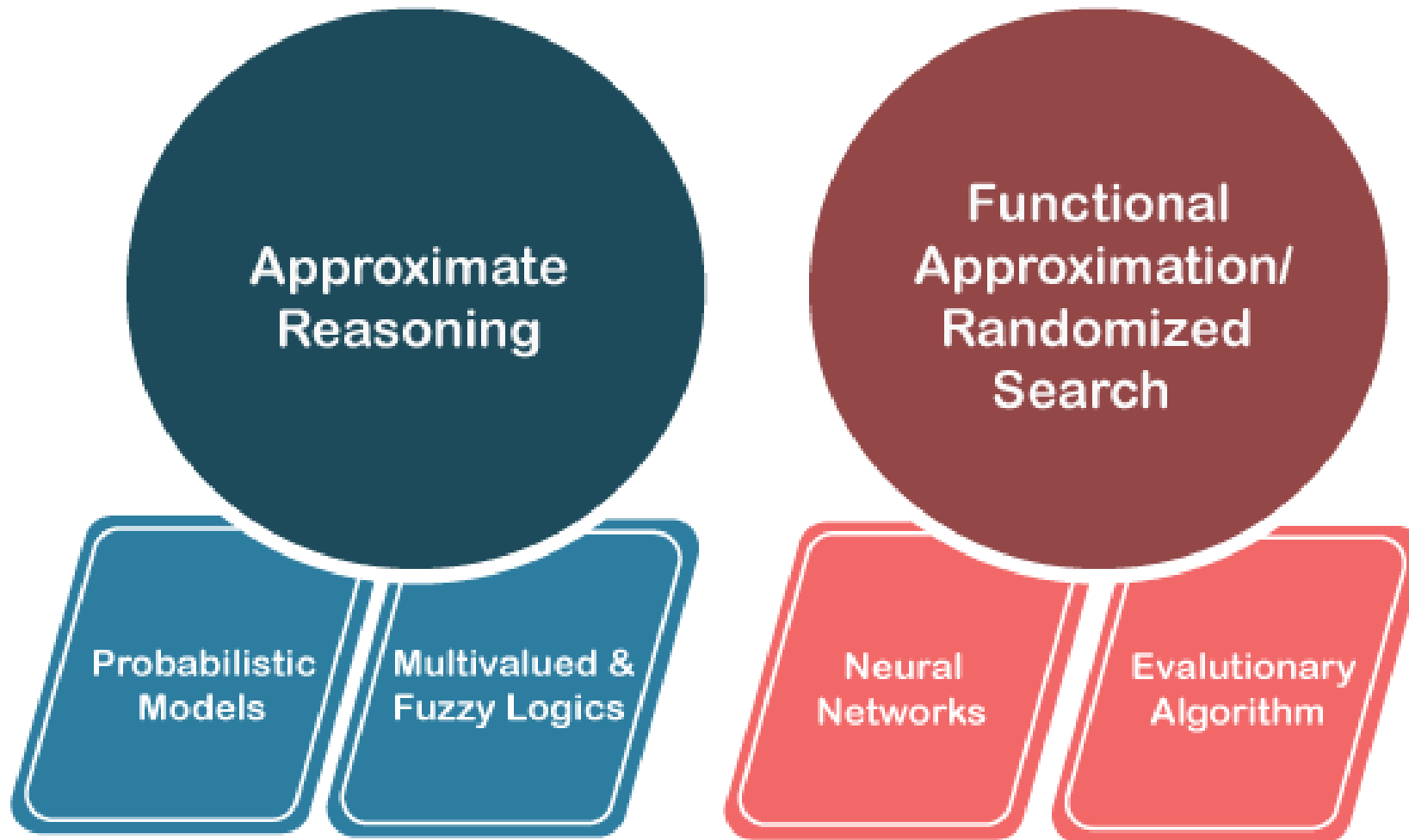
$$\text{SC} = \text{EC} + \text{NN} + \text{FL}$$

Soft Computing	Evolutionary Computing	Neural Network	Fuzzy Logic
-------------------	---------------------------	-------------------	----------------

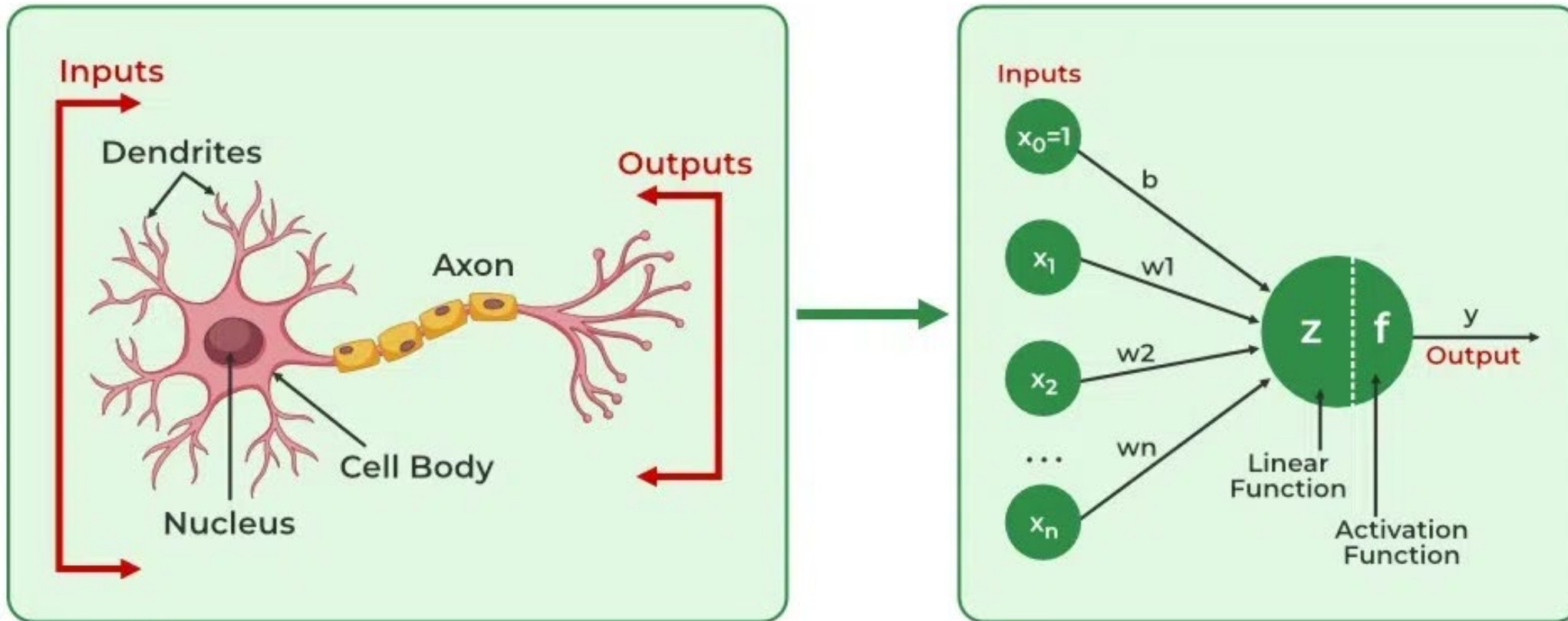
Soft Computing Vs Hard Computing

Hard Computing	Soft Computing
Precisely analytical model	Approximate model
Exact algorithms	Clever algorithms (Heuristic)
Exact input data	Ambiguous/noisy input data
Precision and categoricity	Approximation
Rigidly written programs	Trainable and adaptive
Precise answer	Approximate answer

Components of Soft Computing



Neural Network: Biological Motivation



Neural Network: Biological Neuron

- **Inputs** (red arrow) represent signals received from other neurons.
- **Dendrites**: Branch-like structures that receive input signals.
- **Cell Body (Soma)**: The central part of the neuron that processes inputs.
- **Nucleus**: The core of the neuron, containing genetic material.
- **Axon**: The extended structure that transmits signals to other neurons.
- **Outputs** (red arrow) are signals sent to other neurons

- ## Neural Network: Artificial Neuron (perceptron)
- **Inputs ($x_1, x_2, \dots, x_n$):** Represent numerical values fed into the artificial neuron.
 - **Weights ($w_1, w_2, \dots, w_n$):** Each input is multiplied by a corresponding weight, which determines its importance.
 - **Bias (b):** A constant value added to the weighted sum to adjust the output.
 - **Bias:** type of error occurs due to wrong assumptions
 - **Activation Function (f):** allowing network to learn complex patterns.
 - **Activation Function (f):** Applied to the linear output to introduce non-linearity
 - **Output (y):** The final processed value, determining the neuron's response.

Comparison of Biological Neuron and Artificial Neuron

 Step	 Biological Neuron	 Artificial Neuron (Perceptron)
 1. Input Reception	Dendrites receive signals from other neurons.	Inputs ($x_0, x_1, x_2, \dots, x_n$) are provided to the model.
 2. Signal Processing	Cell body (soma) processes electrical signals.	Each input is multiplied by a corresponding weight (w).
 3. Summation	Signals combine before transmission.	A weighted sum is computed with bias (b).
 4. Activation	The neuron decides whether to fire.	The activation function (f) determines neuron output.
 5. Signal Transmission	Axon sends processed signals to other neurons.	The final output (y) is sent to the next layer.
 6. Learning & Adaptation	Synapse strength changes based on experience.	Weights adjust via backpropagation for learning.

Neural Network

- Neural networks mimic biological brain functions.
- Consist of layers, neurons, and connections.
- Each neuron applies weights and biases.
- Activation functions introduce non-linear properties.
- Outputs help in decision-making processes.
- Process input data using mathematical transformations.

Neural Network: History



Warren McCulloch & Walter Pitts, wrote a paper on how neurons might work; they modeled a simple neural network with electrical circuits.

Nathanial Rochester from the IBM research laboratories led the first effort to simulate a neural network.

John von Neumann suggested imitating simple neuron functions by using telegraph relays or vacuum tubes.

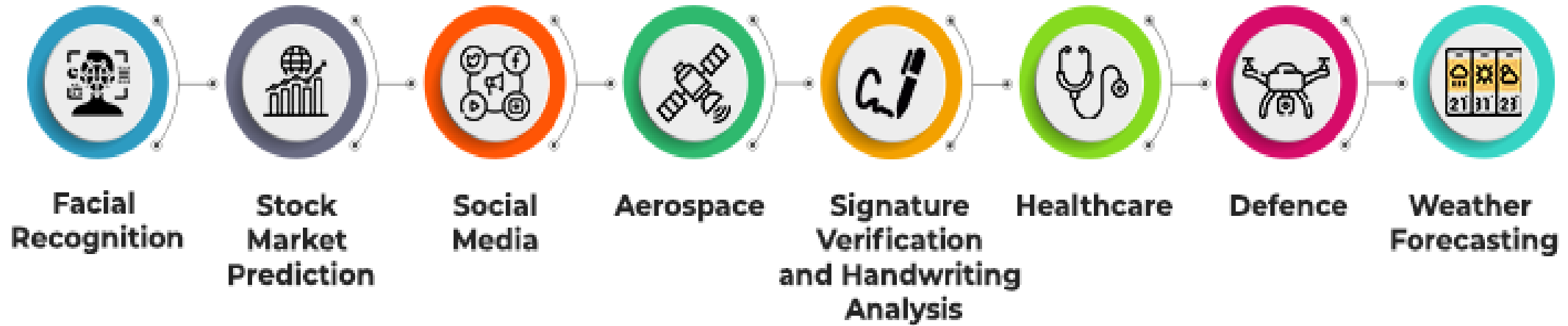
STORY BY DATA

HISTORY OF NEURAL NETWORKS

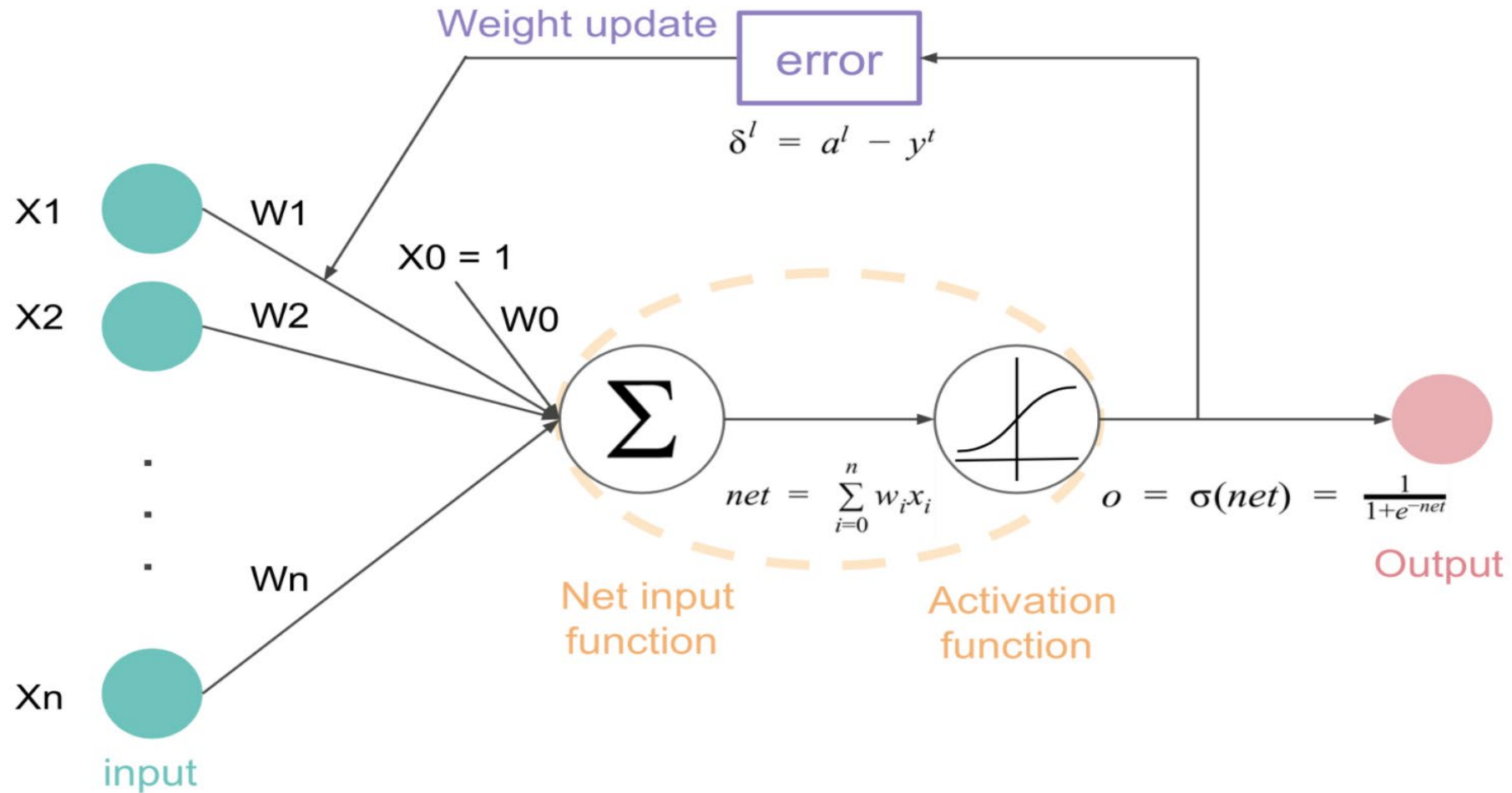
1943-2019



Applications of Neural Networks



Structure and Working of Artificial Neurons



Consist of 3 main parts: input processing, activation function, and weight update (learning process)

Structure and Working of Artificial Neurons

1. Input Processing

- **Inputs** ($X_1, X_2, \dots, X_n$): These are the features or data points fed into the neuron.
- **Weights** ($W_1, W_2, \dots, W_n$): Each input has an associated weight that determines its importance.
- **Bias** ($X_0 = 1, W_0$): A constant input (X_0) with weight (W_0) to shift activation.
- **Net Input Function** (Σ): Computes the weighted sum of inputs:

$$\text{net} = \sum_{i=0}^n w_i x_i$$

This step mimics how biological neurons aggregate incoming signals.

2. Activation Function

- The net input is passed through an **activation function** to introduce non-linearity.
- In this case, the **sigmoid function** is used:

$$o = \sigma(\text{net}) = \frac{1}{1 + e^{-\text{net}}}$$

- The activation function ensures that the output is within a defined range (e.g., between 0 and 1 for sigmoid).

3. Weight Update (Learning Process)

- The **error** is calculated by comparing the predicted output (a^l) with the actual target (y^t):

$$\delta^l = a^l - y^t$$

- This error is used to update the weights using **backpropagation**, improving the model's accuracy over time.

Structure and Working of Artificial Neurons

Final Output

- The processed value is passed to the next layer or used for decision-making.
- The **output neuron** (pink circle) represents the final result after activation.

Summary of the Process

1. Inputs are received with weights.
2. A **weighted sum** is calculated.
3. An **activation function** determines output.
4. The output is produced for further processing.
5. **Backpropagation** updates **weights** to minimize error.

Perceptron Learning Algorithm

- Fundamental supervised learning algorithm used for binary classification problems.
- Based on adjusting weights iteratively to correctly classify data points.

Step 1: Initialize Weights and Bias

- Set all weights w_i (including bias b) to small random values or zero.

Step 2: Compute Net Input

- For each training example $X = (x_1, x_2, \dots, x_n)$, compute the weighted sum (net input function):

$$\text{net} = \sum_{i=1}^n w_i x_i + b$$

Step 3: Apply Activation Function

- Use a step function to determine the output y :

$$y = f(\text{net}) = \begin{cases} 1, & \text{if net} \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

Perceptron Learning Algorithm

Step 4: Update Weights Using Error Correction

- Compare the predicted output y with the actual target y_t and calculate the error:

$$\text{Error} = y_t - y$$

- Update each weight w_i using the formula:

$$w_i = w_i + \eta \cdot \text{Error} \cdot x_i$$

where η is the **learning rate** (a small positive constant).

- Update bias b similarly:

$$b = b + \eta \cdot \text{Error}$$

Step 5: Repeat Until Convergence

- Iterate through the dataset multiple times until all points are correctly classified or a stopping condition is met.

- Hyperparameters are variables that control the training of machine learning models.
- Learning rate (η) is crucial hyperparameter in machine learning that controls how much the model's weights are updated during training.
- η is too high $\rightarrow$ The model converges (produce accurate predictions) too fast
- η is too low $\rightarrow$ The model learns too slowly

Perceptron Learning Algorithm-: Example Calculation

Let's assume we have a perceptron with two inputs and an initial weight set as follows:

- $w_1 = 0.2, w_2 = -0.4, b = 0.5$
- Learning rate $\eta = 0.1$
- Input $(x_1 = 1, x_2 = 0)$ with target output $y_t = 1$

Step 1: Compute Net Input

$$\text{net} = (0.2 \times 1) + (-0.4 \times 0) + 0.5 = 0.7$$

Since $0.7 \geq 0$, the output $y = 1$, which matches y_t , so no weight update is needed.

If there was an error, we would update weights using the rule from **Step 4**.

Perceptron Learning Algorithm-: Example Calculation

Given Data

- Initial Weights:

$$w_1 = 0.3, \quad w_2 = -0.2, \quad b = 0.4$$

- Learning Rate:

$$\eta = 0.1$$

- Input:

$$x_1 = 1, \quad x_2 = 1$$

- Target Output:

$$y_t = 0$$

Step 1: Compute Net Input

Using the net input function:

$$\text{net} = (w_1 \cdot x_1) + (w_2 \cdot x_2) + b$$

Substituting values:

$$\text{net} = (0.3 \times 1) + (-0.2 \times 1) + 0.4 = 0.3 - 0.2 + 0.4 = 0.5$$

Perceptron Learning Algorithm-: Example Calculation

Step 2: Apply Activation Function

Using the step activation function:

$$y = \begin{cases} 1, & \text{if net} \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

Since $\text{net} = 0.5$ (which is ≥ 0), the perceptron predicts $y = 1$.

Step 3: Compute Error

$$\text{Error} = y_t - y = 0 - 1 = -1$$

Since there is an error, we need to update the weights and bias.

Perceptron Learning Algorithm-: Example Calculation

Step 4: Update Weights and Bias

Using the perceptron learning rule:

$$w_i = w_i + \eta \cdot \text{Error} \cdot x_i$$

$$b = b + \eta \cdot \text{Error}$$

Updating weights:

$$w_1 = 0.3 + (0.1 \times -1 \times 1) = 0.3 - 0.1 = 0.2$$

$$w_2 = -0.2 + (0.1 \times -1 \times 1) = -0.2 - 0.1 = -0.3$$

$$b = 0.4 + (0.1 \times -1) = 0.4 - 0.1 = 0.3$$

Final Updated Values

- New Weights:

$$w_1 = 0.2, \quad w_2 = -0.3$$

- New Bias:

$$b = 0.3$$

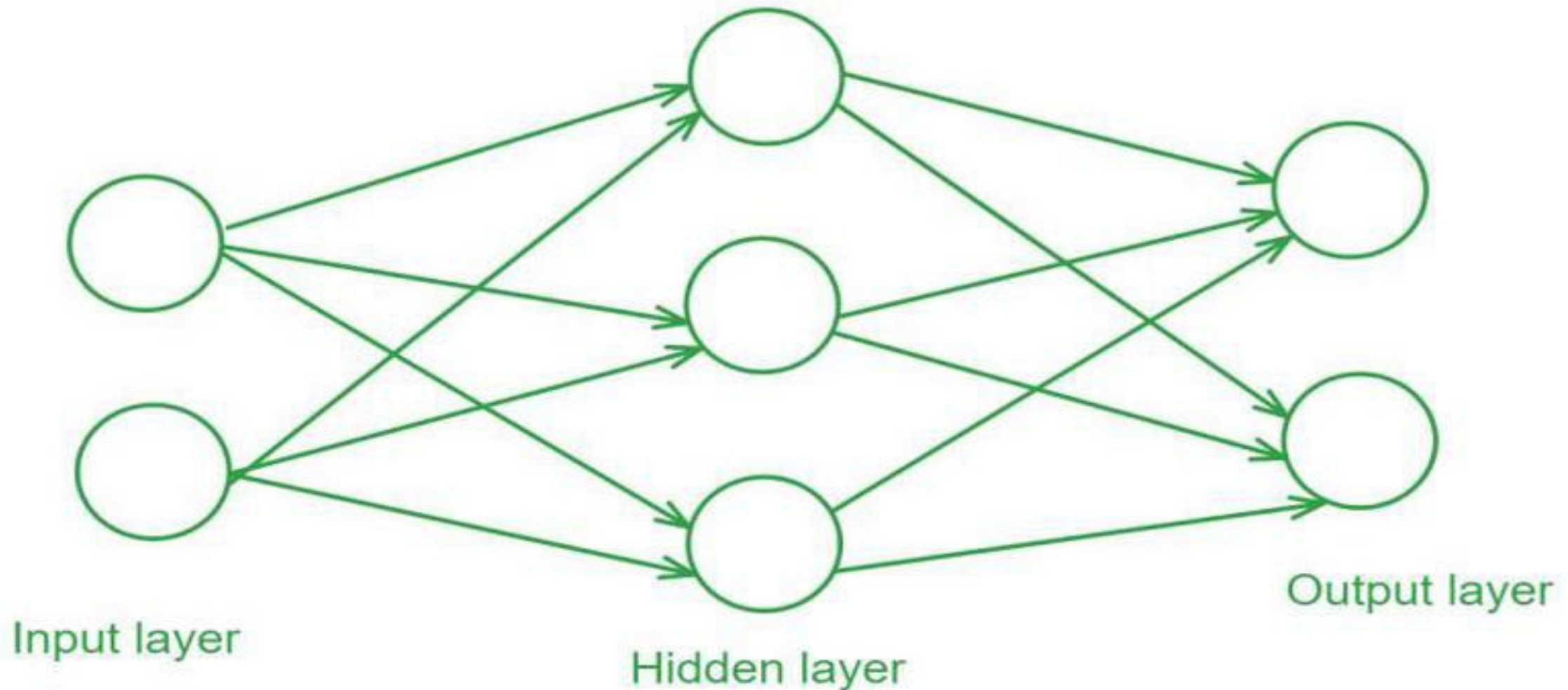
Architecture and Working of Feedforward Neural Networks

- ANN is mathematical model that can learn and used in ML.
- Feedforward Neural Network (FNN) is the simplest type of ANN where information flows only in one direction.
- Flows from I/P to O/P without looping.
- FNN consists of 3 layers:
- I/P layer, Hidden layer, O/P layer
- loss function measures how well the neural network's predictions match the actual target values.

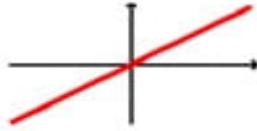
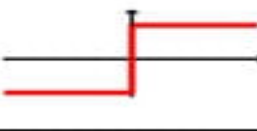
Architecture and Working of Feedforward Neural Networks

- **Input Layer:** Receives raw data or features ($x_1, x_2, \dots, x_n$).
- No computations are performed in this layer.
- **Hidden layer :** Contains multiple neurons that process input data.
- Each neuron applies weighted sum of inputs followed by an activation function.
- **Output Layer :** The number of neurons depends on the type of problem
- Binary classification: 1 neuron (sigmoid activation).
- Multi-class classification: n neurons (softmax activation).
- Regression: 1 neuron (linear activation).

Architecture and Working of Feedforward Neural Networks

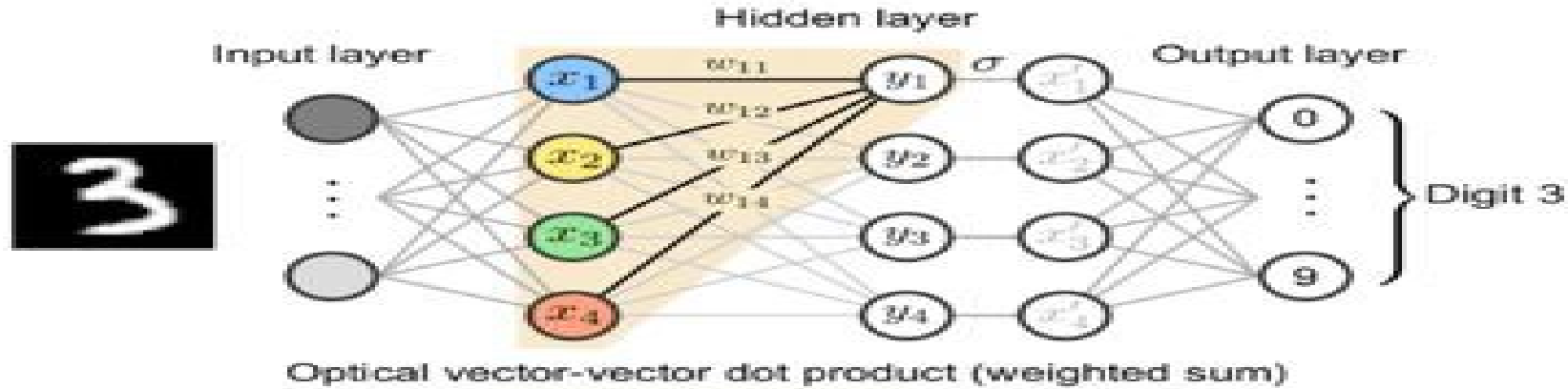


Activation Functions

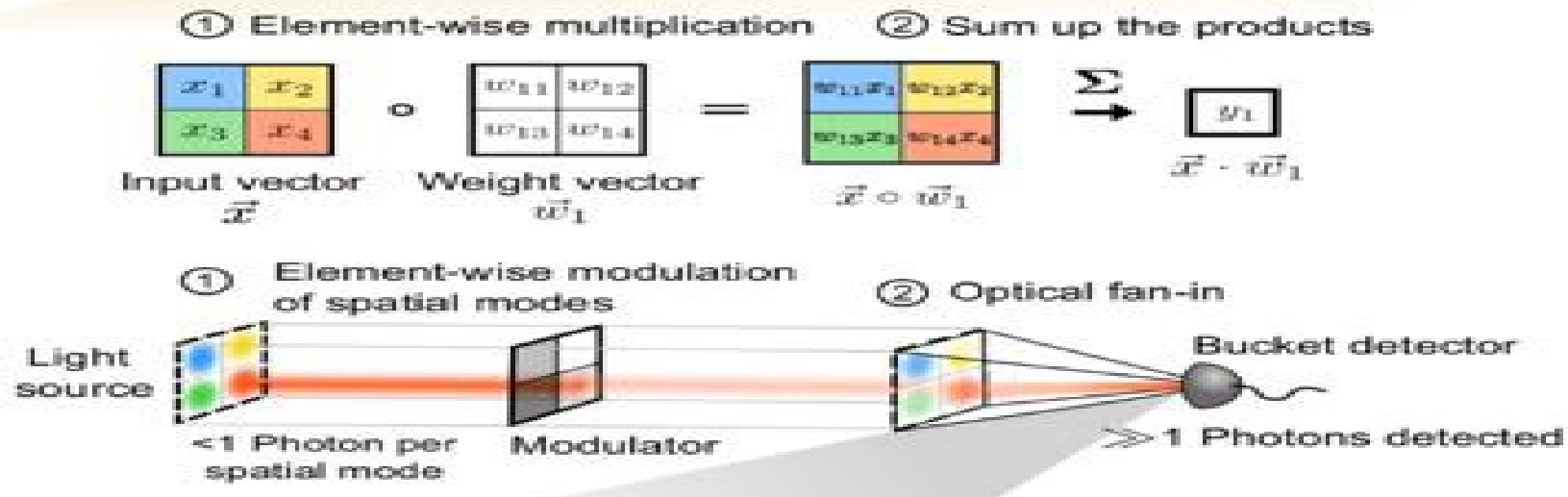
Activation Function	Equation	Example	1D Graph
Linear	$\phi(z) = z$	Adaline, linear regression	
Unit Step (Heaviside Function)	$\phi(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\phi(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise Linear	$\phi(z) = \begin{cases} 0 & z \leq -1/2 \\ z + 1/2 & -1/2 \leq z \leq 1/2 \\ 1 & z \geq 1/2 \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multilayer NN	
Hyperbolic Tangent (tanh)	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\phi(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

Case study: Practical Problem-solving using Neural Networks

a



b



Case study: Practical Problem-solving using Neural Networks

(a) Neural Network Structure for Digit Recognition

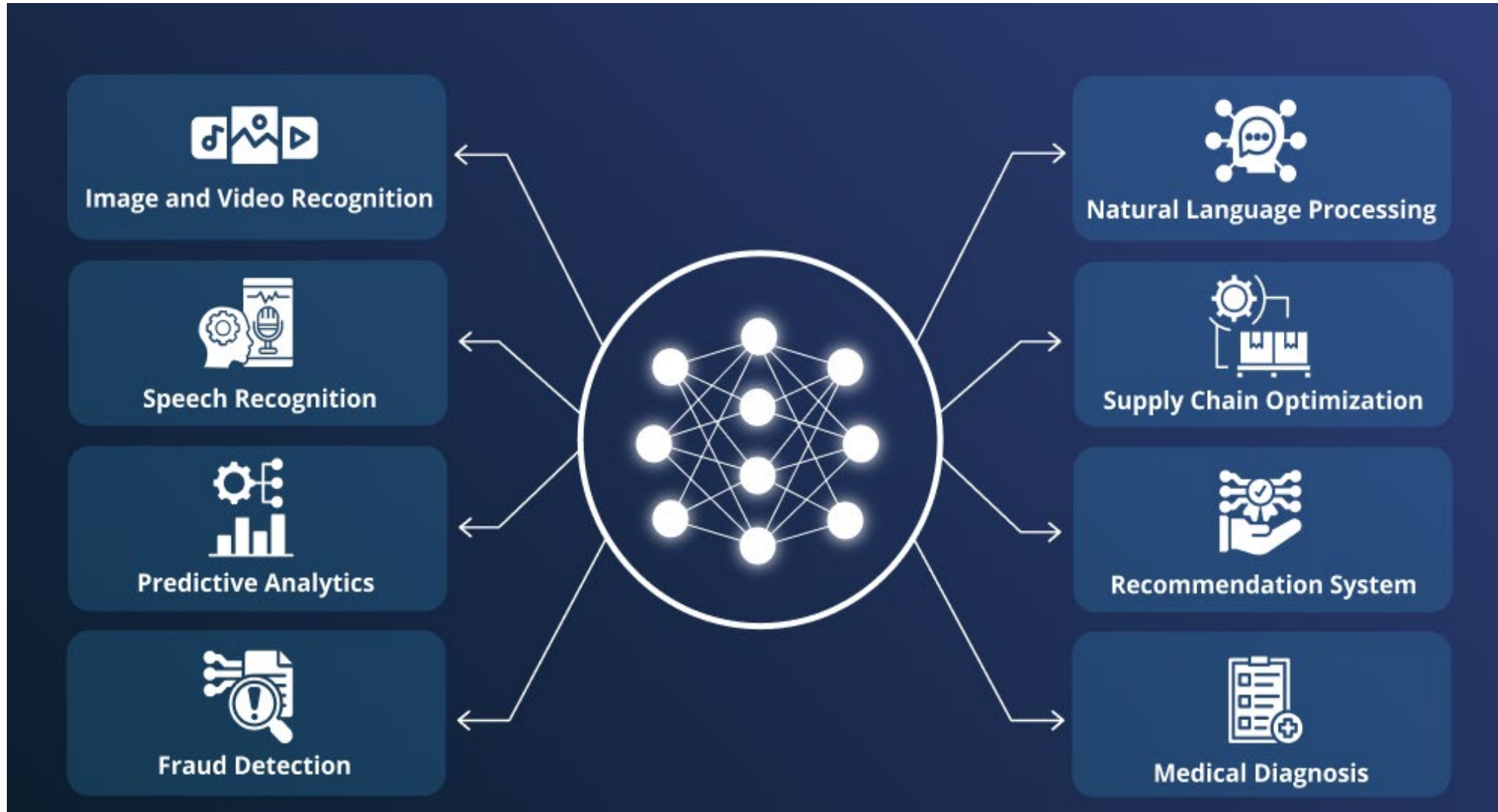
- Represents a feedforward neural network used for handwritten digit recognition.
- Input layer consists of pixel values from a grayscale (shades of gray, without any color) image of the digit "3."
- Hidden layer performs weighted summation and applies an activation function (σ).
- The output layer has 10 neurons (0-9), where the neuron corresponding to "3" has the highest activation.
- **Key Concept:** The weighted sum of inputs and weights is computed using a dot product, followed by an activation function to introduce non-linearity.

Case study: Practical Problem-solving using Neural Networks

(b) Optical Computation of Neural Network Operations

- Explains how optical components can perform vector-matrix multiplication, a key operation in neural networks.
- Step 1: Element-wise multiplication
 - Each input value x_i is multiplied by the corresponding weight w_{ij} .
- Step 2: Summation of products
 - The results are summed to obtain the neuron activation y_1 .
- Optical Interpretation
 - A light source represents input data.
 - A modulator applies weights to the input data in a spatially parallel manner.
 - A bucket detector aggregates the weighted signals, simulating a neuron summation.

Case study: Practical Problem-solving using Neural Networks

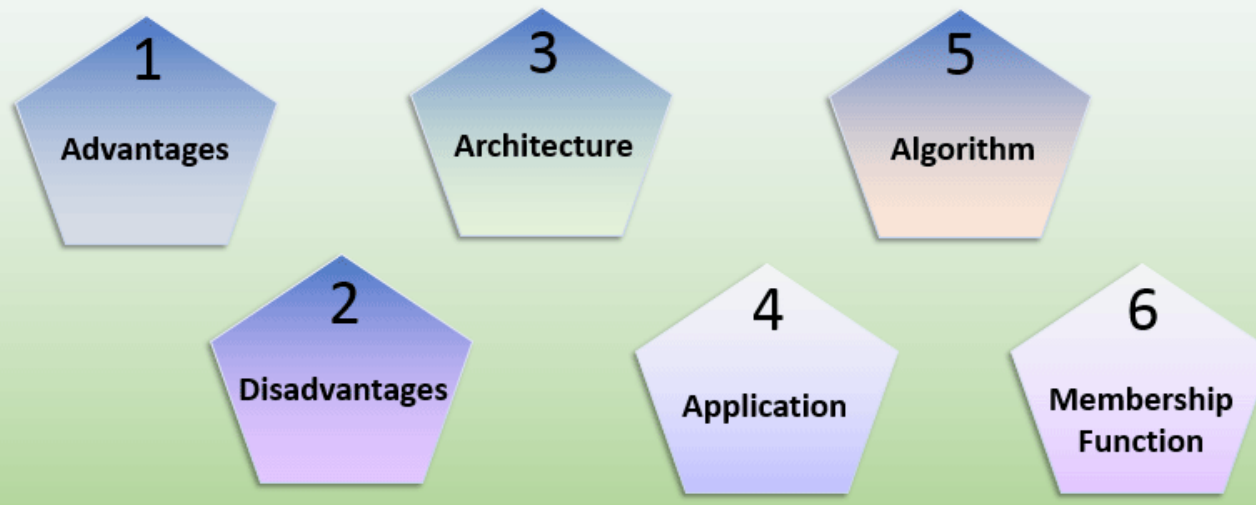


Lovely Professional University
Department of Computer Science & Engineering

Presentation on

INT254-Foundations of Machine Learning

Fuzzy Logic System



Presented By:

Dr. Premananda Sahu

Department of Machine Learning

Topics to be covered...

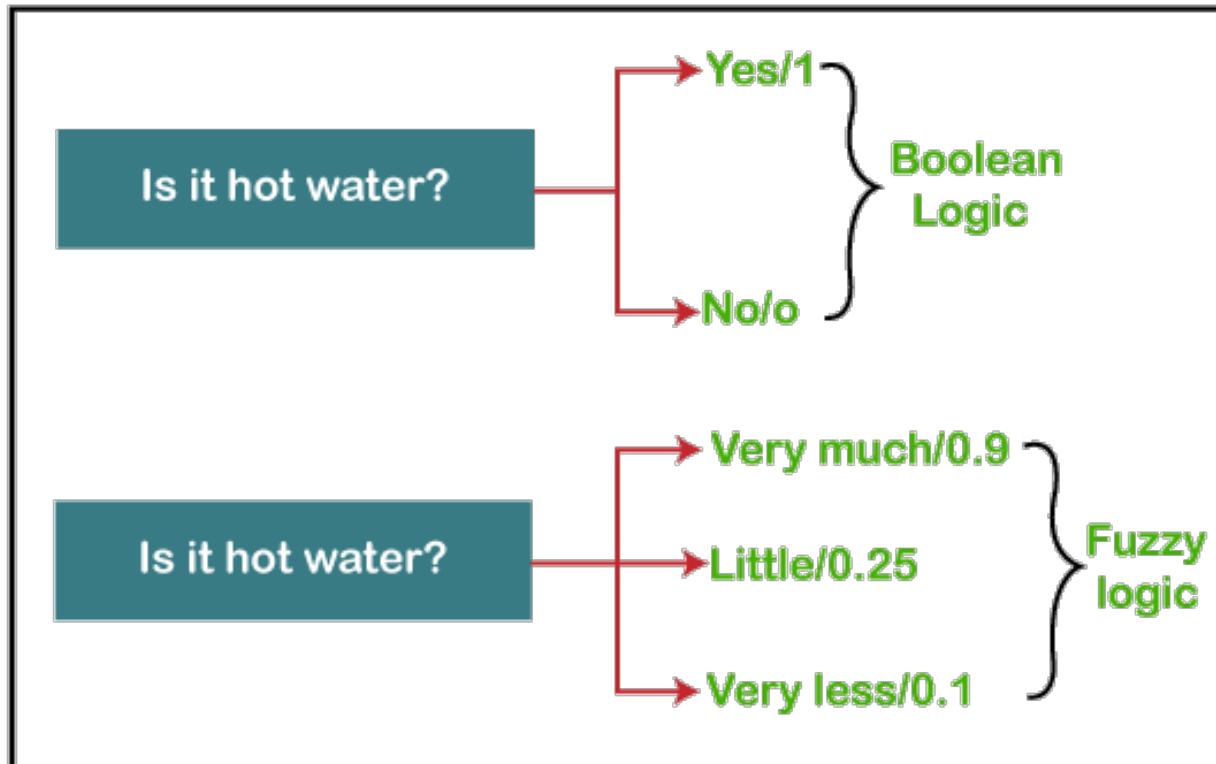
- Fuzzy Logic: Basic Definition and Terminology
- Set-theoretic Fuzzy Operations
- Fuzzy Sets and Operations on Fuzzy Sets
- Fuzzy Relations, Fuzzy Rules and Fuzzy Reasoning
- Fuzzy Inference System
- Fuzzification and Defuzzification Methods
- Fuzzy-based Expert System

Fuzzy Logic: Basic Definition

- Fuzzy Sets: Instead of crisp sets, fuzzy sets allow partial membership with a degree between 0 and 1.
- partial membership means set to a certain degree.
- Membership Functions: Define how each input belongs to a fuzzy set.
- Fuzzy Rules: If-Then rules that govern decision-making (e.g., "If temperature is high, then fan speed is fast").
- Fuzzification: Converts crisp input values into fuzzy values.
- Defuzzification: Converts fuzzy outputs back into crisp values for real-world applications

Fuzzy Logic: Basic Definition

- Handles the **concept of partial truth**.
- Allows **reasoning with imprecise or uncertain data**.



Fuzzy Logic: Terminology

- Fuzzy Set
- Membership Function
- Linguistic Variables
- Fuzzy Rules (IF-THEN Rules)
- Fuzzification
- Inference
- Defuzzification
- Fuzzy Operators (AND, OR, NOT)
- Alpha-cut (α -cut)

Fuzzy Logic: Terminology

- Linguistic Variables: Variables described in natural language instead of numerical values.
- Example: Instead of saying "temperature = 30°C", we say:
- Temperature is "Warm"
- Inference Engine: Applies fuzzy rules to make decisions.
- Example: Input: Temperature = 35°C
- Fuzzy rules apply:
- "If temperature is high, set AC to fast mode."
- Output: AC speed 75%.

Fuzzy Logic: Terminology

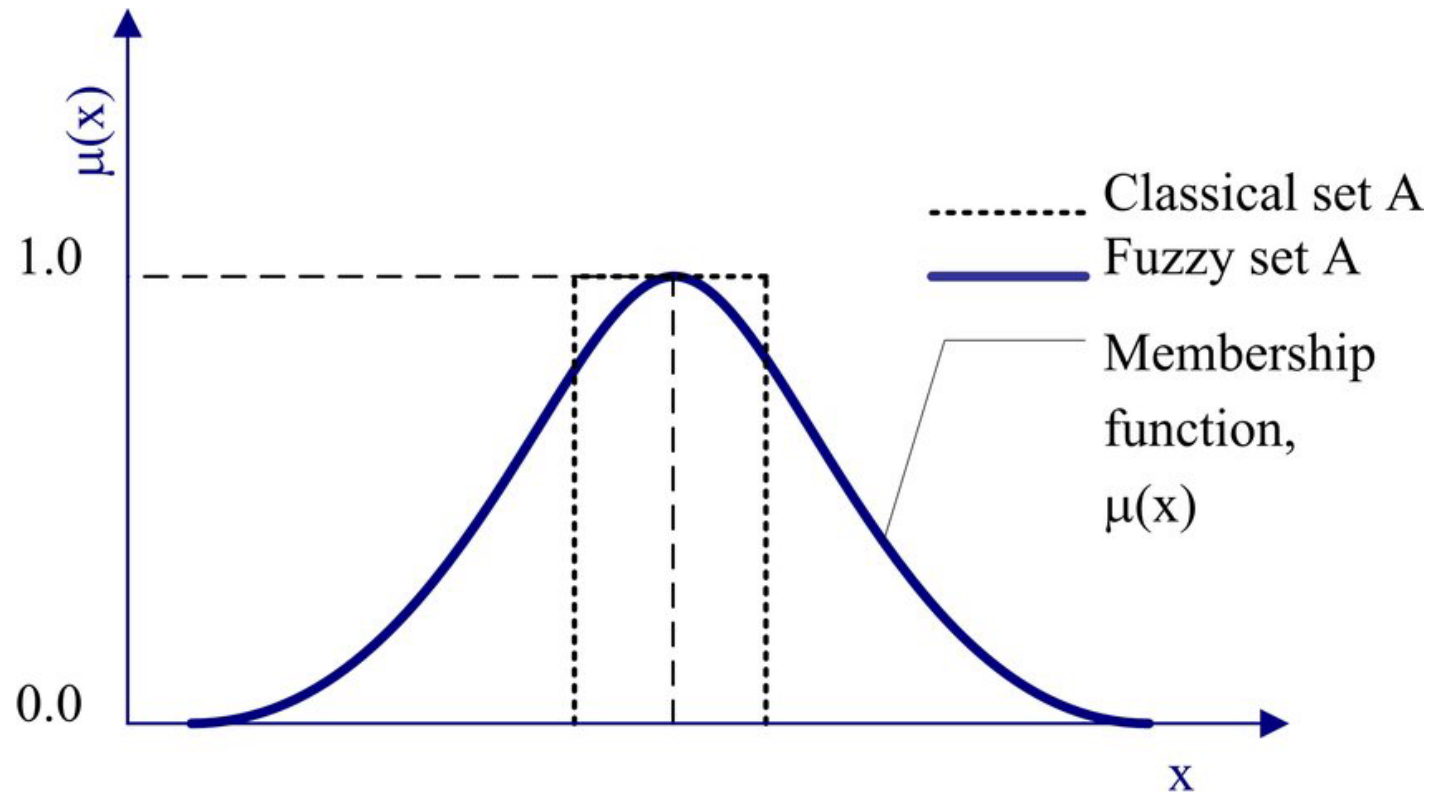
- Fuzzy Operators (AND, OR, NOT) Used to combine fuzzy values.
- Example: AND (min rule): "Hot" (0.7) AND "Humid" (0.6) $\rightarrow \text{Min}(0.7, 0.6) = 0.6$
- OR (max rule): "Hot" (0.7) OR "Humid" (0.6) $\rightarrow \text{Max}(0.7, 0.6) = 0.7$
- NOT: "Hot" (0.7) $\rightarrow 1 - 0.7 = 0.3$ (Not Hot)
- Alpha-cut (α -cut) A method for converting a fuzzy set into a crisp set by keeping only elements with membership $\geq \alpha$.
- Example ($\alpha = 0.5$): A fuzzy set "Warm" has: 20°C (0.3) $\rightarrow$ Removed
- 25°C (0.6) $\rightarrow$ Kept
- 30°C (0.8) $\rightarrow$ Kept
- Final α -cut set: {25°C, 30°C}

Set-theoretic Fuzzy Operations

- An object has a numeric “degree of membership”.
- Normally, between 0 and 1
 - 0 means the object is not in the set
 - 1 means the object is fully inside the set
 - In between means the object is partially in the set

Fuzzy Sets

- Collection of elements with **varying degrees of membership**.
- Ranging from **0 (not a member)** to **1 (fully a member)**.



compares classical and fuzzy sets, illustrating how fuzzy sets allow gradual transitions rather than strict boundaries

Operations on Fuzzy Sets

- Union (OR)

- **Definition:** Combines two fuzzy sets
- **Formula:** $\mu_{\{A \cup B\}}(x) = \max(\mu_{A(x)}, \mu_{B(x)})$

- Intersection (AND)

- **Definition:** Finds common elements in two fuzzy sets
- **Formula:** $\mu_{\{A \cap B\}}(x) = \min(\mu_{A(x)}, \mu_{B(x)})$

- Complement (NOT)

- **Definition:** Reverses the membership degree
- **Formula:** $\mu_{NOT A}(x) = 1 - \mu_{A(x)}$

Set-theoretic Fuzzy Operations

$$A = \{1/a, 0.3/b, 0.2/c, 0.8/d, 0/e\}$$

$$B = \{0.6/a, 0.9/b, 0.1/c, 0.3/d, 0.2/e\}$$

Complement:

$$A' = \{0/a, 0.7/b, 0.8/c, 0.2/d, 1/e\}$$

Union:

$$A \cup B = \{1/a, 0.9/b, 0.2/c, 0.8/d, 0.2/e\}$$

Intersection:

$$A \cap B = \{0.6/a, 0.3/b, 0.1/c, 0.3/d, 0/e\}$$

Fuzzy Relations, Fuzzy Rules and Fuzzy Reasoning

- Fuzzy Relations:

- Allowing **degrees of association**
- Represented as membership values in $[0, 1]$.

A **fuzzy relation** represents a relationship between multiple fuzzy sets. It generalizes the concept of classical relations by allowing degrees of association between elements instead of strict "yes" or "no" membership.

Mathematical Representation:

A fuzzy relation is typically represented as a **fuzzy matrix** or a **membership function** that assigns a value between 0 and 1 to each pair of elements from two sets.

For example, if we have two fuzzy sets:

- **Set X** = {Students}
- **Set Y** = {Grades}

A fuzzy relation **R** might define how well a student understands a subject. The membership value $\mu_R(x, y)$ represents the degree to which a student **x** belongs to grade **y**.

Fuzzy Relations, Fuzzy Rules and Fuzzy Reasoning

Example:

If the fuzzy relation matrix is:

Student →	Poor	Average	Good	Excellent
Alice	0.1	0.5	0.8	0.2
Bob	0.4	0.6	0.7	0.3

- Alice has **0.8** membership in the "Good" category, meaning she is likely a good student.
- Bob has **0.6** membership in the "Average" category, meaning he is moderately good.

Types of Fuzzy Relations:

1. Reflexive - Every element is at least somewhat related to itself ($\mu_R(x, x) > 0$).
2. Symmetric - If $\mu_R(x, y) = \mu_R(y, x)$, the relation is symmetric (e.g., "is as tall as").
3. Transitive - If x is related to y and y is related to z, then x should be related to z to some degree.

Fuzzy Rules

Fuzzy rules are **IF-THEN** statements used to model human reasoning in fuzzy logic systems. These rules define relationships between input and output fuzzy sets.

Structure of a Fuzzy Rule:

A general fuzzy rule looks like this:

IF (Condition) THEN (Consequence)

- **IF part (Antecedent):** Defines a fuzzy condition based on input variables.
- **THEN part (Consequent):** Defines the fuzzy output.

Example:

A fuzzy rule for air conditioning:

- **IF temperature is high THEN fan speed is fast**
- **IF temperature is moderate THEN fan speed is medium**
- **IF temperature is low THEN fan speed is slow**

Here, "high", "moderate", and "low" are fuzzy linguistic terms, not exact values.

Fuzzy Reasoning

Fuzzy reasoning is the process of making decisions based on fuzzy rules. It transforms fuzzy inputs into fuzzy outputs by applying fuzzy inference techniques.

Steps in Fuzzy Reasoning:

1. **Fuzzification** - Convert crisp (numerical) inputs into fuzzy values.
2. **Apply Fuzzy Rules** - Use the rule base to determine how the inputs relate to the outputs.
3. **Aggregation** - Combine all rule outputs into a single fuzzy output.
4. **Defuzzification** - Convert the fuzzy output back into a crisp value.

Example of Fuzzy Reasoning:

Problem: Determine the braking force of a car based on speed and road slipperiness.

- **Input 1 (Speed):** "Slow", "Medium", "Fast"
- **Input 2 (Road Condition):** "Dry", "Wet", "Icy"
- **Output (Braking Force):** "Low", "Moderate", "High"

Fuzzy Inference System(FIS)

- framework that processes input data using fuzzy logic principles to make intelligent decisions.
- takes crisp (numerical) inputs, processes them using **fuzzy rules**, and produces a crisp or fuzzy output.
- widely used in decision-making, expert systems, and artificial intelligence.
- E.S designed to solve complex problems by reasoning.
- represented mainly as if–then rules rather than through conventional procedural programming code.

Components of FIS

- **Fuzzification** – Converts crisp inputs into fuzzy sets.
- **Fuzzy Rule Base** – A set of **IF-THEN** rules that define the system's behavior.
- **Inference Engine** – Applies fuzzy logic principles to evaluate rules and determine outputs.
- **Aggregation** – Combines all rule outputs into a final fuzzy set.
- **Defuzzification** – Converts the fuzzy output into a crisp numerical value.

Step-by-Step Working of FIS

- **Step 1:** Fuzzification (Convert Crisp Inputs to Fuzzy Values)
- The FIS first takes crisp numerical inputs and converts them into fuzzy values using membership functions.
- Example:
- Input = Temperature = 30°C
- Membership functions:
- "Cold" → 0.2
- "Warm" → 0.7
- "Hot" → 0.1
- This means that 30°C is mostly warm, slightly cold, and not very hot.

Step-by-Step Working of FIS

- **Step 2: Rule Evaluation (Apply Fuzzy Rules)**
- The FIS then applies IF-THEN rules from the fuzzy rule base.
- Example fuzzy rules for an air conditioning system:
 - Rule 1: IF temperature is hot, THEN fan speed is high
 - Rule 2: IF temperature is warm, THEN fan speed is medium
 - Rule 3: IF temperature is cold, THEN fan speed is low
- Since 30°C is 70% warm and 20% cold, both Rule 2 and Rule 3 will be partially activated.

Step-by-Step Working of FIS

- Step 3: Aggregation (Combine Rule Outputs)
- The system combines all the outputs from the activated fuzzy rules.
- If Rule 2 (medium fan speed) is triggered with a 0.7 membership,
- and Rule 3 (low fan speed) is triggered with a 0.2 membership,
- then both outputs will be combined accordingly.

Step-by-Step Working of FIS

Step 4: Defuzzification (Convert Fuzzy Output to Crisp Output)

- The final step converts the fuzzy output into a single crisp value.
- Common defuzzification methods:
 - Centroid Method (Weighted average) – Most commonly used.
 - Max Membership Method – Chooses the most dominant fuzzy set.
 - Mean of Maximum – Averages the max values.

For example, if the fan speed values are:

- Low = 20%
- Medium = 70%
- High = 10%
- Then, the final crisp fan speed might be calculated as 45% of max speed.

Types of FIS

1. Mamdani FIS (Most Common, Intuitive for Humans)

Uses fuzzy IF-THEN rules where both the input and output are fuzzy sets.

Example Rule:

IF temperature is hot THEN fan speed is high (both input and output are fuzzy).

Requires defuzzification to convert fuzzy output into a crisp value.

Types of FIS

2. Sugeno FIS (Computationally Efficient)

- The output is a mathematical function instead of a fuzzy set.
- Example Rule:
 - IF temperature is hot THEN fan speed = $0.8 \times \text{temperature} + 5$
- No need for defuzzification (output is already crisp).

Comparison	Mamdani FIS	Sugeno FIS
Output Type	Fuzzy set	Crisp
Computation Speed	Slower	Faster
Interpretability	More intuitive	More mathematical
Usage	Expert systems, AI	automation

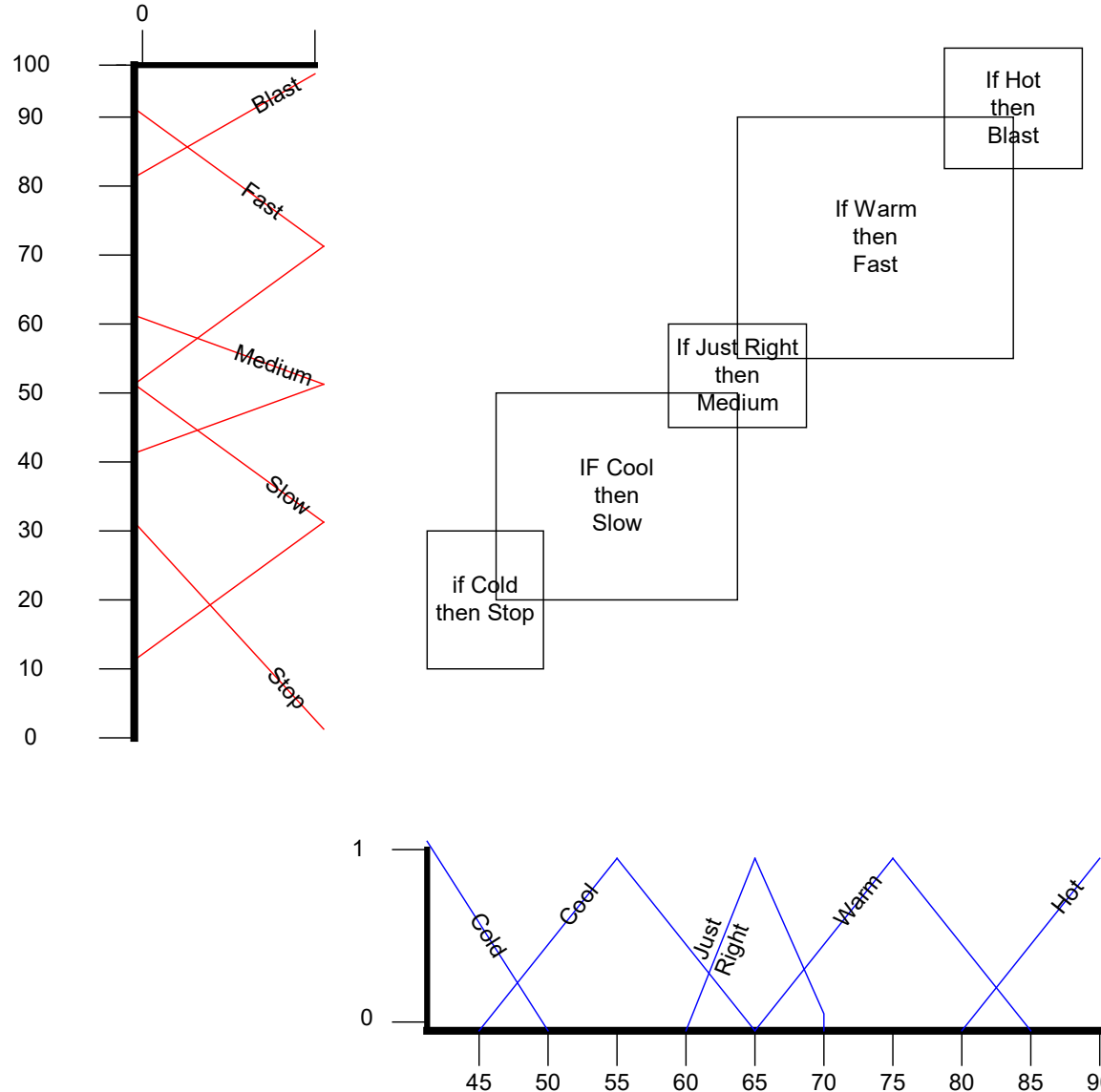
Application of FIS

- ✓ **Industrial Automation** – Washing machines, air conditioners
- ✓ **Medical Diagnosis** – Predicting diseases based on symptoms.
- ✓ **Stock Market Prediction** – Handling uncertainty in stock prices.
- ✓ **Self-Driving Cars** – Adjusting speed based on traffic conditions.
- ✓ **Weather Forecasting** – Predicting rain or temperature variations.

Fuzzy based Expert System

- uses fuzzy logic to handle uncertainty and imprecise information.
- extends traditional expert systems by incorporating degrees of truth rather than binary true/false values.
- Example-:
 - IF temperature is high AND cough is severe, THEN possibility of infection is high.
 - Input: Temperature = 101°F, Cough Severity = Moderate
 - Fuzzy Output: Probability of infection = 75%
 - The fuzzy system provides probabilistic results rather than absolute decisions, making it more reliable in uncertain situations.

Fuzzy based Expert System



Fuzzy based Expert System

Bottom-Right Graph (Input Membership Functions):

- Defines the fuzzy input variable, likely temperature.
- x-axis represents temperature values (ranging from about 45 to 90).
- y-axis represents the degree of membership (0 to 1).
- temperature is classified into five fuzzy sets:
 - Cold (45-55)
 - Cool (50-60)
 - Just Right (55-70)
 - Warm (65-80)
 - Hot (75-90)
- Each fuzzy set has triangular membership functions that overlap.

Fuzzy based Expert System

Top-Right Diagram (Fuzzy Rules):

- This section represents the fuzzy rule base, which dictates how input temperature determines the system's output.
- The rules are structured as IF-THEN statements:
 - If Cold $\rightarrow$ Stop
 - If Cool $\rightarrow$ Slow
 - If Just Right $\rightarrow$ Medium
 - If Warm $\rightarrow$ Fast
 - If Hot $\rightarrow$ Blast
- These rules describe how the system adjusts fan speed based on temperature.

Fuzzy based Expert System

Left Graph (Output Membership Functions):

- This graph defines the fuzzy output variable, likely fan speed.
- The y-axis represents different fan speeds (Stop, Slow, Medium, Fast, Blast).
- The fan speed is divided into five fuzzy sets, corresponding to the rules:
 - Stop (0%)
 - Slow (20-40%)
 - Medium (40-60%)
 - Fast (60-80%)
 - Blast (80-100%)
- The red triangular membership functions indicate how output fan speed varies.

How Fuzzy Logic works in fans or HVAC

- widely used in fan speed control and HVAC (Heating, Ventilation, and Air Conditioning) systems because it efficiently handles uncertainty and imprecise data.
- Instead of using a rigid on/off or stepwise control, fuzzy logic allows smooth and adaptive regulation based on temperature, humidity.

Example Fuzzy Rules for Fan Speed:

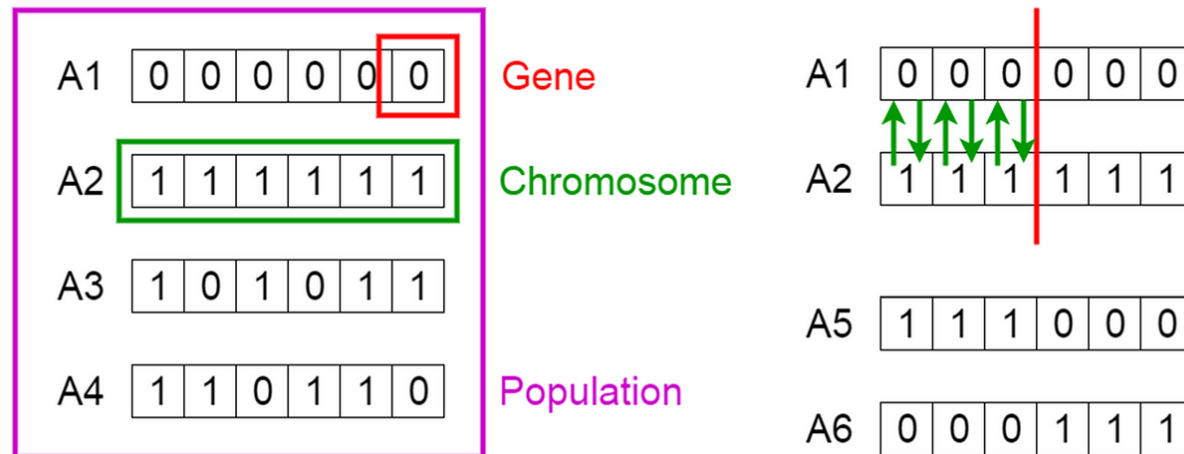
- - 1 If Temperature is Cold, then Fan Speed is Stop.
 - 2 If Temperature is Cool, then Fan Speed is Slow.
 - 3 If Temperature is Just Right, then Fan Speed is Medium.
 - 4 If Temperature is Warm, then Fan Speed is Fast.
 - 5 If Temperature is Hot, then Fan Speed is Blast.
- These rules mimic human decision-making in adjusting the fan or HVAC system gradually

Lovely Professional University
Department of Computer Science & Engineering

Presentation on

INT254-Foundations of Machine Learning

Genetic Algorithms



Presented By:

Dr. Premananda Sahu

Department of Machine Learning

Topics to be covered...

- Genetic algorithms: Introduction to Genetic Algorithms
- Biological Inspiration
- Genetic Operators
- Fitness Function
- Working of Genetic Algorithm
- Applications of Genetic Algorithm

Genetic Algorithms

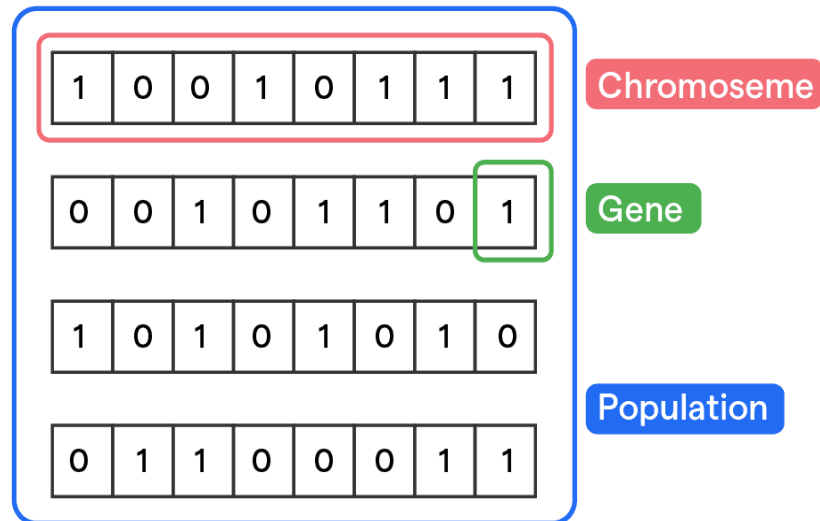
- Type of optimization algorithm inspired by the principles of natural selection and genetics.
- family of evolutionary algorithms and are used to find optimal solutions for complex problems.
- **Concepts-:**
 - Population: group of potential solutions (individuals) for the given problem.
 - Each individual is represented as a chromosome (a set of parameters or genes).
 - Fitness Function: A function that evaluates how good an individual is in solving the problem.
 - The better the fitness score, the higher the chance of survival.

Genetic Algorithms

- Selection: The process of choosing individuals from the population for reproduction based on their fitness.
- Crossover (Recombination): combines two parent solutions to create one or more offspring.
- New solutions (offspring) are created by combining the genetic material (genes) of the parent solutions.
- Mutation: introduces random changes in the genetic material of the offspring.
- help explore the solution space and prevent premature convergence.

Introduction to Genetic Algorithms

Genetic Algorithms



Chromosome: Represented at the top with a red border. It is a sequence of binary digits, here shown as **10010111**, symbolizing a potential solution to a problem.

Gene: The middle section with a green border highlights individual binary digits, like **1**, representing a smaller unit or part of the solution within the chromosome.

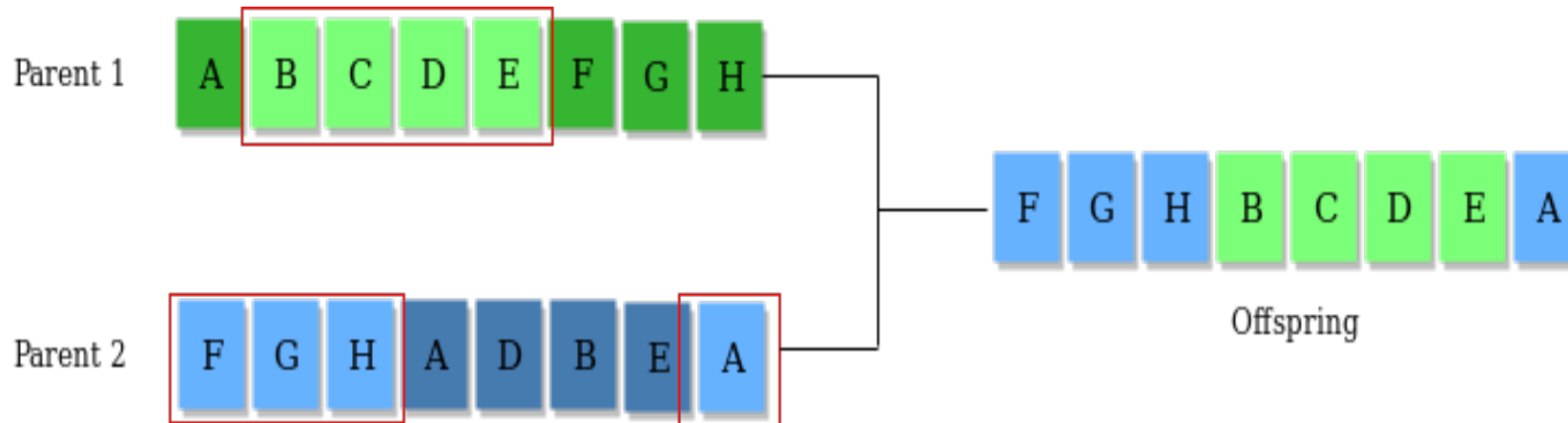
Population: The bottom section enclosed in a blue border shows multiple binary sequences, such as:

- **10010111**
- **00101101**
- **10101010**
- **011100011**

Genetic Operators

- **Selection Operator**
- **Crossover Operator**
- represents mating between individuals.
- Two individuals are selected using selection operator and crossover sites are chosen randomly.
- the genes at these crossover sites are exchanged thus creating a completely new individual (offspring).

• Ex-:



Genetic Operators

- Single-Point Crossover-: chosen on both parent chromosomes.
- The genetic material before the point is taken from one parent, and after the point, it's taken from the other parent.

Example:

- Parent 1: **101|0101**
- Parent 2: **011|1100**
- Offspring: **1011100** and **0110101**

Genetic Operators

- Two-Point Crossover-:
- Two points are selected on the parent chromosomes.
- The segment between these points is swapped.

Example:

- Parent 1: **10|111|001**
- Parent 2: **01|000|101**
- Offspring: **10000101** and **01111001**

Genetic Operators

- Uniform Crossover-:
- Each gene in the offspring is randomly taken from one of the parents, based on a probability distribution.

Example:

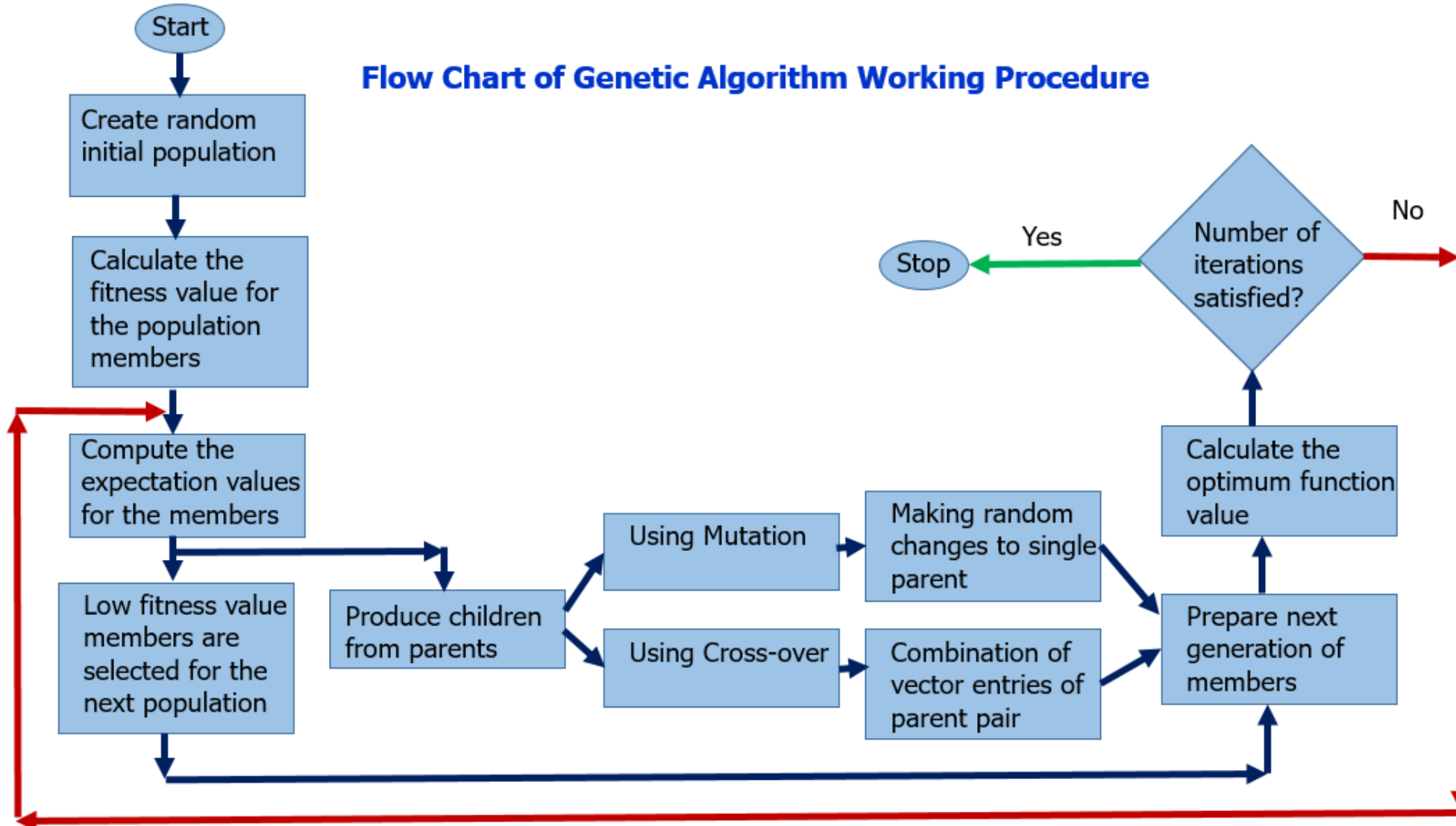
- Parent 1: **110010**
- Parent 2: **101101**
- Offspring: **111101** (random selection of genes)

Fitness Function

- Evaluates **how well a candidate solution or offspring** solves the problem.
- **Numerical Output:** Assigns a fitness score, indicating the quality of the solution.
- fitness score is numerical value assigned to each solution (chromosome) to evaluate its quality
- **Guides Evolution:** Probability of an individual being selected for reproduction.

Working of Genetic Algorithm

Flow Chart of Genetic Algorithm Working Procedure



Working of Genetic Algorithm

1. Start: The algorithm begins its execution.

2. Create Random Initial Population:

- A group of potential solutions (population) is randomly generated. Each solution in this population is typically represented as a chromosome.

3. Calculate the Fitness Value for the Population Members:

- Each solution in the population is evaluated to determine how well it solves the given problem. This is done by calculating its fitness value using a predefined fitness function.

4. Compute the Expectation Values for the Members:

- Based on their fitness values, expectation values are calculated for each member. These expectation values represent the likelihood of each solution being selected for reproduction in the next generation.

Working of Genetic Algorithm

5. Select Low Fitness Value Members for the Next Population:

- Members with lower fitness values are selected to contribute to the next generation. This is part of the selection process, which is critical for evolution.

6. Produce Children from Parents:

- New solutions (children or offspring) are created by combining and modifying the genetic material of selected parent solutions. This is achieved using two techniques:
 - **Mutation:** Random changes are introduced into a single parent's genetic material to create diversity.
 - **Crossover:** Genetic material from two parents is combined to produce new solutions with traits from both.

Working of Genetic Algorithm

7. Prepare Next Generation of Members:

- The newly created children, along with the parents that were selected, form the next generation of solutions.

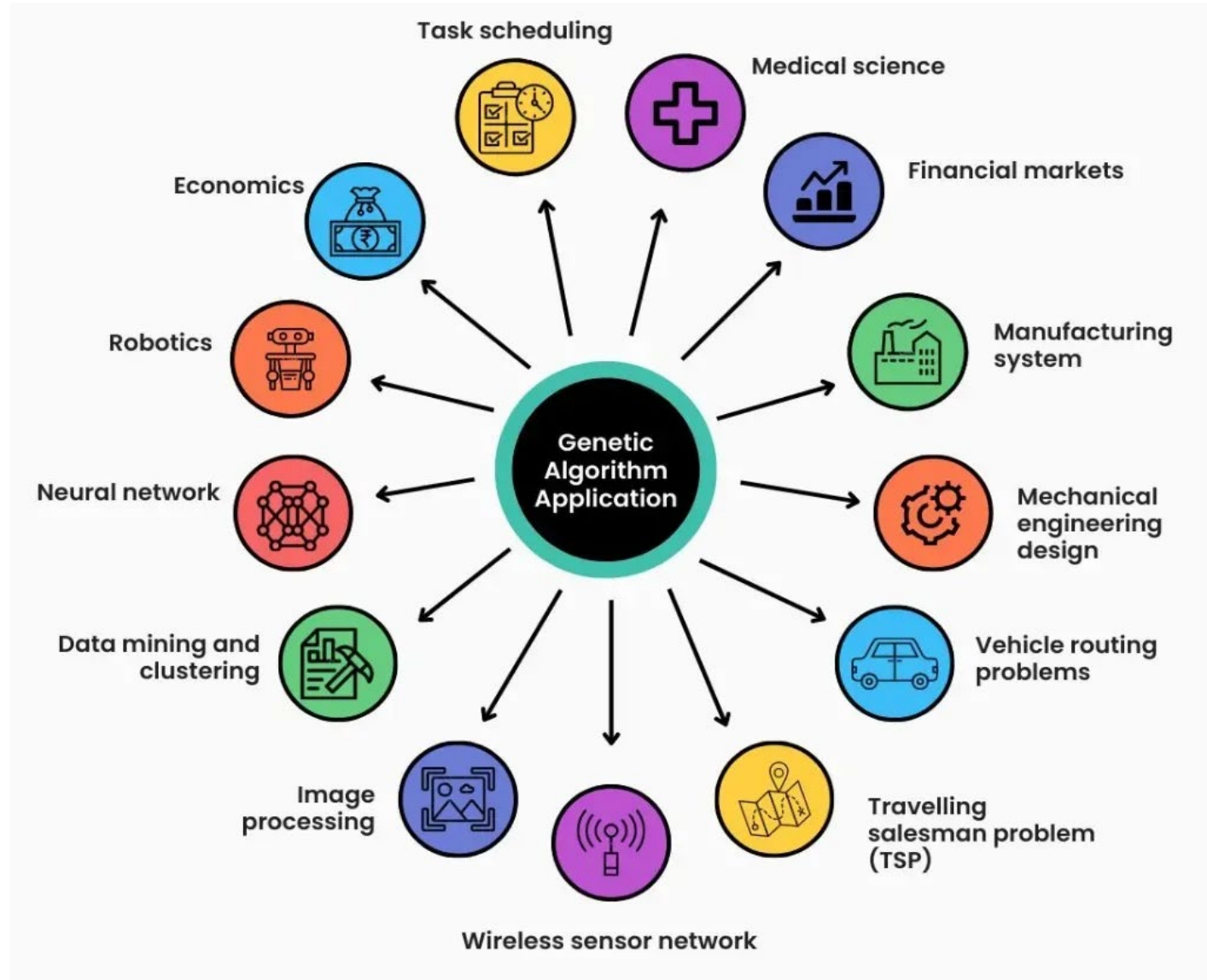
8. Calculate the Optimum Function Value:

- The best solution in the current generation is evaluated, and its fitness is checked to determine if it meets the optimization criteria.

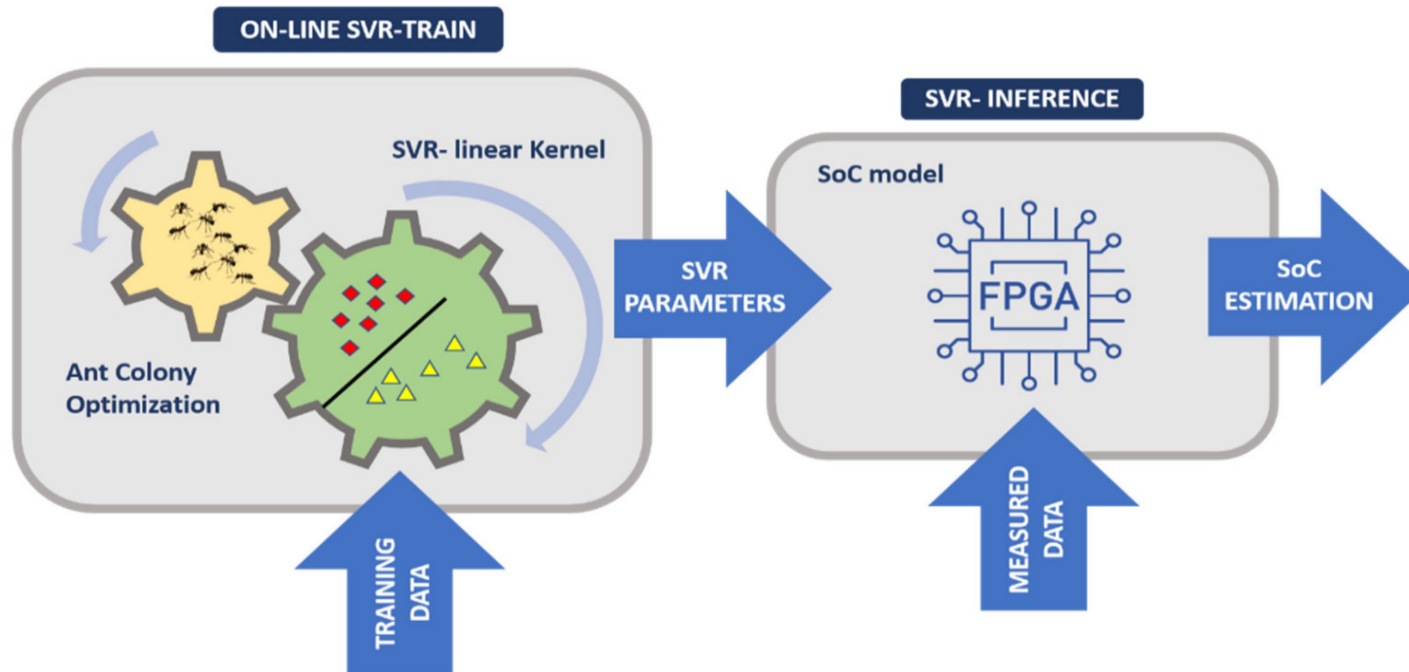
9. Check if the Number of Iterations is Satisfied:

- This is a termination condition:
 - If the desired number of iterations or the optimal solution is achieved, the algorithm **stops**.
 - Otherwise, it loops back to **Step 3**, repeating the process with the new generation.

Applications of Genetic Algorithm



Lovely Professional University
Department of Computer Science & Engineering
Presentation on
INT254-Foundations of Machine Learning



Presented By:
Dr. Premananda Sahu
Department of Machine Learning

Topics to be covered...

- Swarm Intelligence
- Ant Colony Optimization
- Swarm Intelligence in Bees
- Cuckoo Search
- Firefly Algorithm
- Crow Search Algorithm
- Hybrid Wolf-Bat Algorithm
- Whale Search Algorithm
- Grasshopper Optimization

Swarm Intelligence (SI)

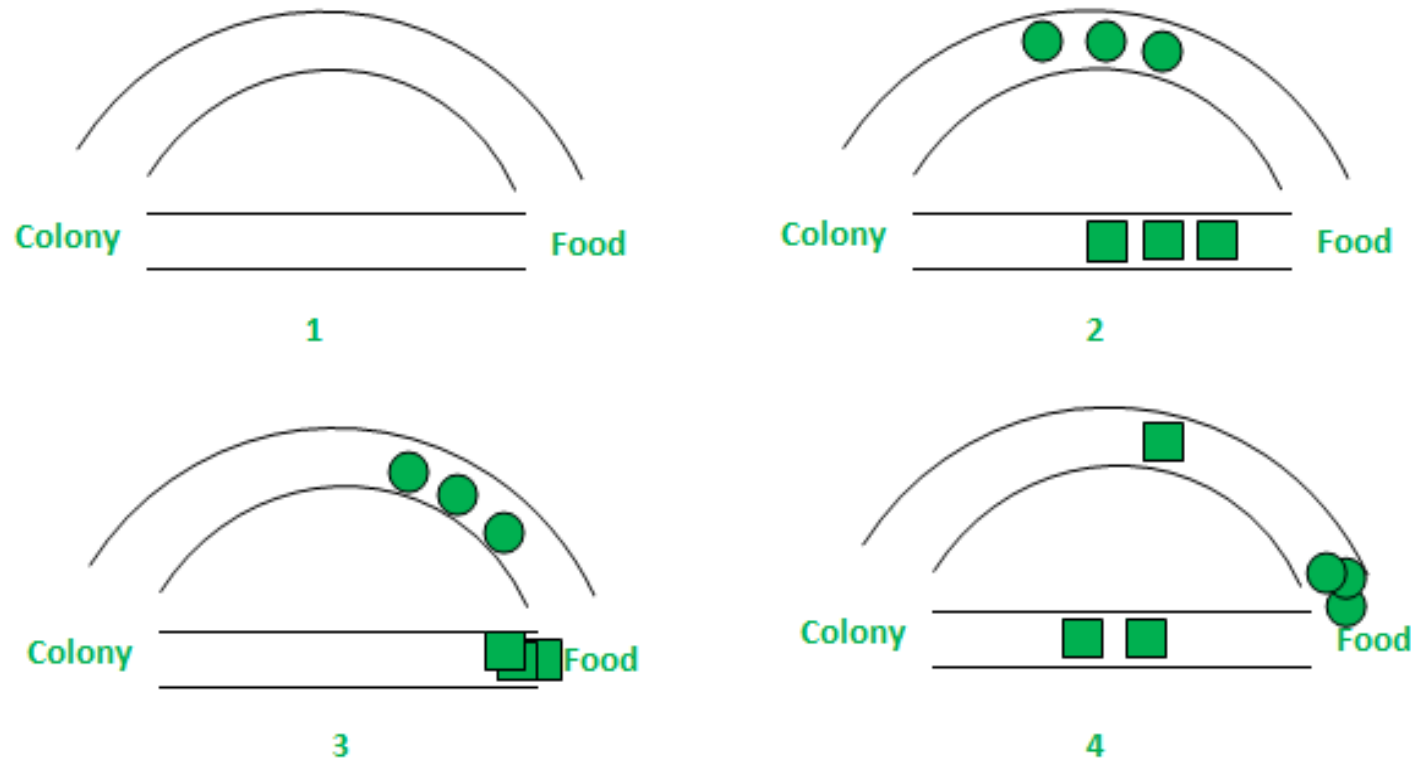
- Inspired by the collective behavior of decentralized, self-organized systems.
- Used to solve optimization and search problems.
- made up of simple agents (like birds, ants, or bees), interacting with each other and their environment.
- Decentralization: each individual makes decisions.
- Self-organization: Complex collective behavior appears from simple rules followed by individuals.
- Simple rules: Each agent follows a basic set of rules.
- Adaptability: The swarm can adjust to changing environments.

Swarm Intelligence (SI)

- As a form of AI, swarm intelligence comprises a network of endpoint devices capable of generating and processing data at the source.
- Example-: self-driving vehicles able to gather and process traffic data could share it with other vehicles in the network to allow them to react to changing traffic conditions.

Ant Colony Optimization (ACO)

- Nature-inspired optimization algorithm
- Used to solve complex combinatorial optimization problems.
- Optimization algorithm inspired by the behavior of ants.



Ant Colony Optimization (ACO)

- 1. This stage represents the beginning, where ants start to leave the colony in search of food.
- There are no pheromone trails laid down yet.
- 2. Ants randomly explore different paths.
- As they find food, they return to the colony while laying pheromones to mark the route.
- 3. The shortest path has the most pheromone because more ants have used it successfully.
- This positive feedback loop strengthens the trail.
- 4. The ants have collectively discovered and committed to the most efficient path between the colony and the food source.

Steps of Ant Colony Optimization (ACO)

- Initialization: We start by placing the ants on the starting point.
- Movement: Each ant selects a point to move to based on a probabilistic function that takes into account the pheromone level and the heuristic information.
- The heuristic information can be thought of as a measure of how good a particular point is.
- Updating pheromone trails: The pheromone trail on each edge is updated based on the quality of the solution found by the ant that traversed that edge.
- Termination: We stop the algorithm after a certain number of iterations or when a satisfactory solution is found.

Swarm Intelligence in Bees

inspired by the behavior of honey bees.

used to solve optimization problems.

works by simulating a colony of bees that are searching for the best location for a new store.

evaluating different locations and sharing their findings with the rest of the colony.

Artificial Bee Colony (ABC)

- Honey bees' searching behavior inspired the ABC
- Population based algorithm, the position of a food source represents a possible solution to the optimization problem
- Nectar amount of a food source corresponds to the quality (fitness) of the associated solution.
- Three types of bees: employed bees, onlooker bees, and scout bees

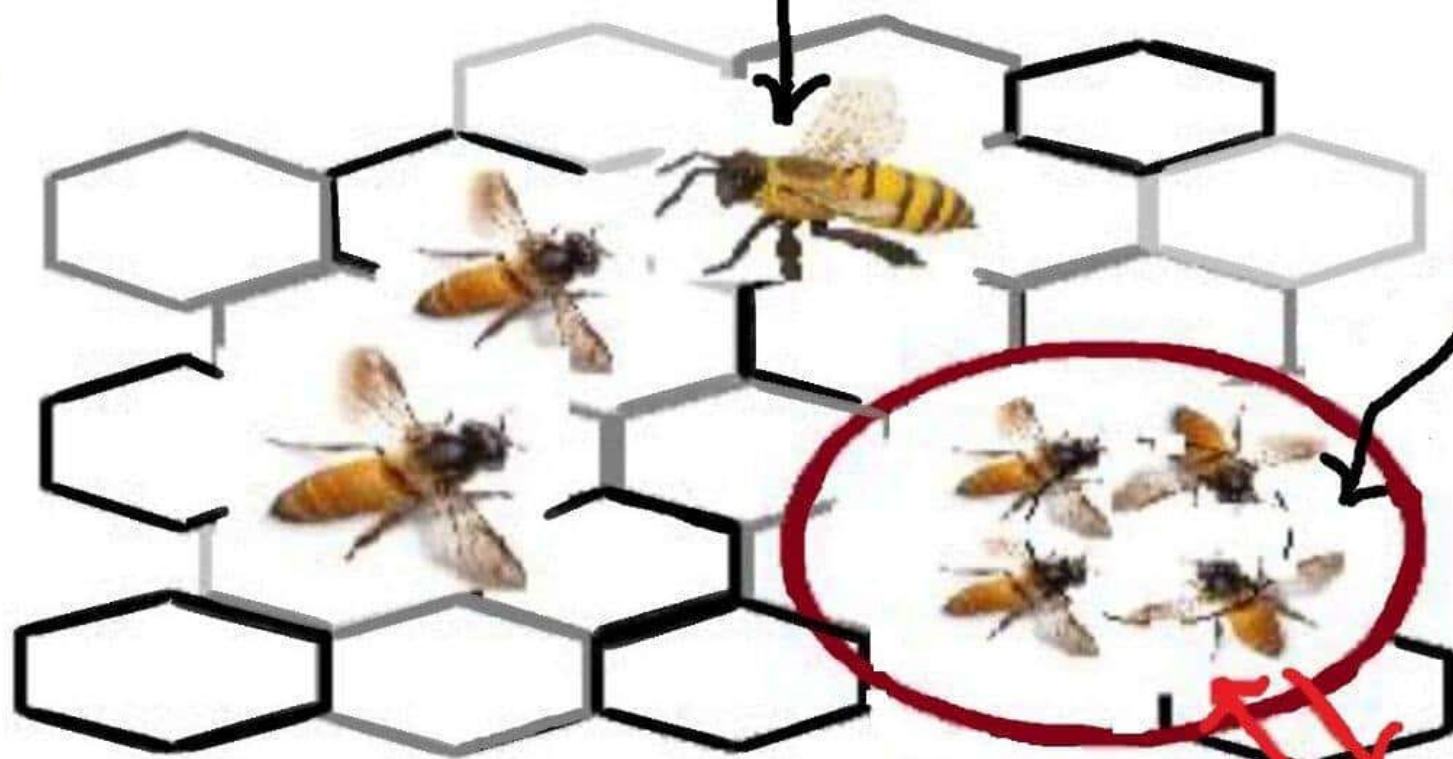
scout bee

onlooker bee

dancing area

employed bee

food source



Artificial Bee Colony (ABC)

- **Scout Bee:** Responsible for exploring the search space randomly to discover new food sources (potential solutions).
- This helps maintain diversity in the population and prevents premature convergence.
- **Employed Bee:** Associated with a specific food source, each employed bee performs a local search in the neighborhood of its current solution and evaluates its quality.
- It then shares this information with the onlooker bees.
- **Onlooker Bee:** Located within the store, onlooker bees observe the dances of employed bees in the dancing area and probabilistically select food sources based on their fitness.
- They then further exploit these selected sources.

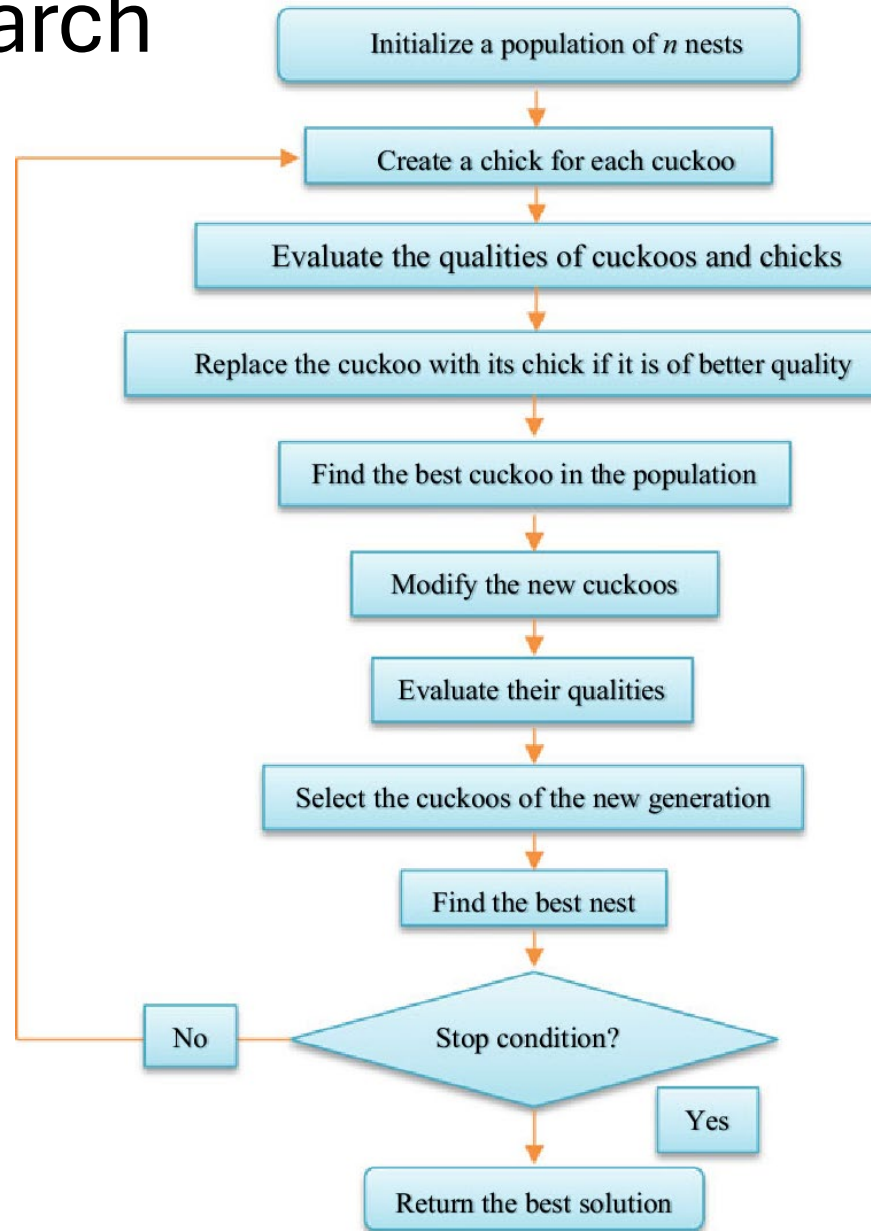
Artificial Bee Colony (ABC)

- **Dancing Area:** This is the communication zone where employed bees advertise the quality of food sources through a waggle dance, influencing the decision-making of onlooker bees.
- **Food Source:** Represents a candidate solution to the optimization problem.
- The quality or fitness of the solution corresponds to the amount of nectar at the food source.
- It improves solutions through a balance of exploration (scout bees) and exploitation (employed and onlooker bees), effectively navigating the search space.

Cuckoo Search

- Inspired by the cuckoo birds.
- Cuckoo birds lay their eggs in the nests of the other host birds.
- Each cuckoo can only lay one egg at a time.
- Then deposit it in a randomly selected nest.
- Secondly, the best nests with high-quality eggs will be passed down to future generations.
- Thirdly, the number of available host nests is fixed.

Cuckoo Search



Cuckoo Search

- 1. Start with a set of potential solutions (nests).
- Establish a population of nests, each possibly containing a solution to the optimization problem.
- 2. Generate a new solution (chick) based on the existing one (cuckoo).
- Cuckoo lays eggs in random nests, hoping they get raised.
- 3. Assess the fitness (or performance) of each solution.
- Compare how good the parent and the offspring are.
- 4. Update the population by keeping the better solution.
- Improve the solution pool by natural selection.

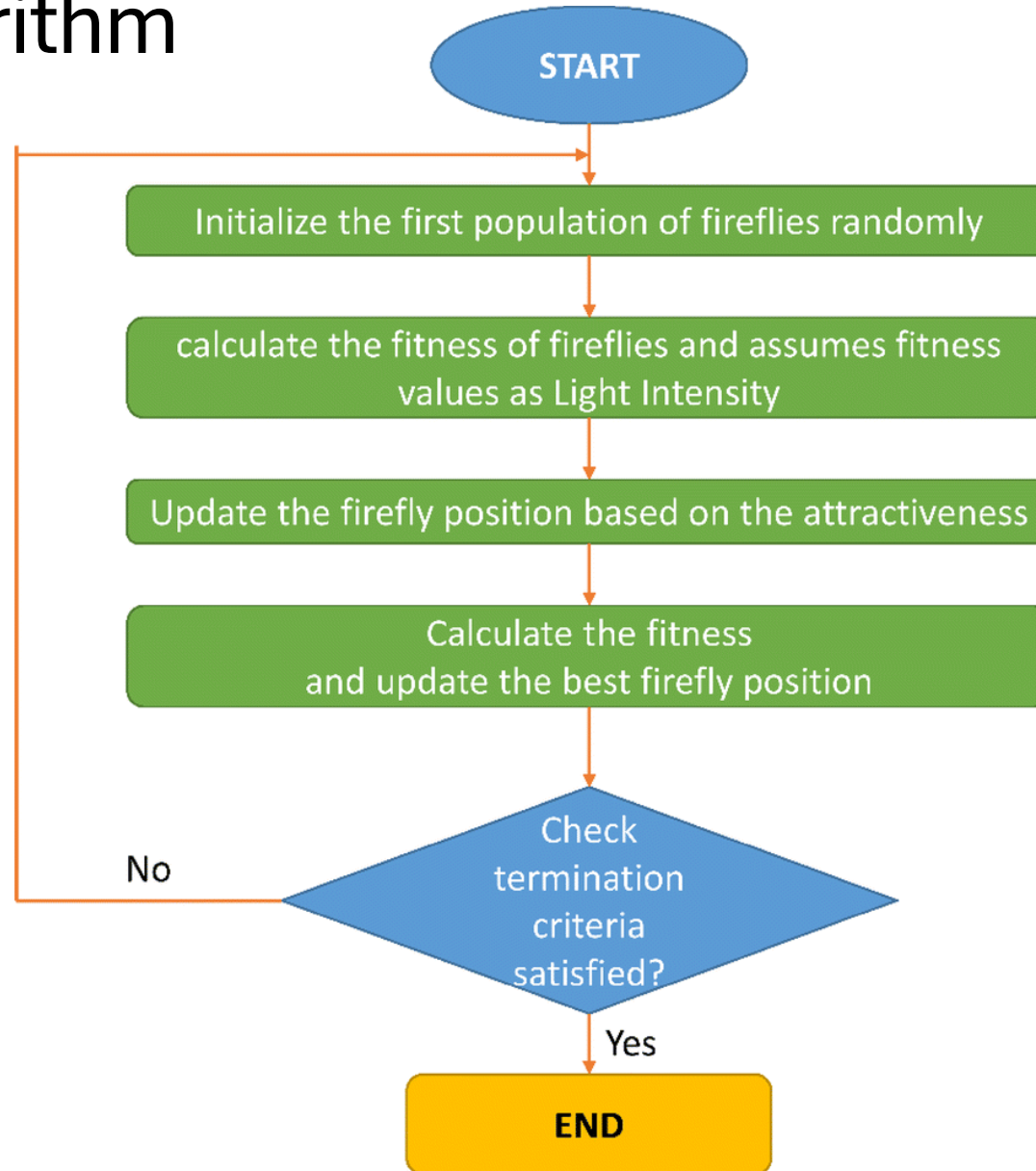
Cuckoo Search

- 5. Identify the top-performing solution so far.
- Guides the evolution of the next generation.
- 6. Apply variation techniques (e.g., Lévy flights or mutations).
- Lévy flights are certain probability distribution with random directions.
- 7. Explore the search space more effectively.
- Reconsider the newly modified cuckoos (solutions).
- 8. Choose the best candidates for the next iteration.
- Ensure that the next generation is composed of higher quality solutions.

Cuckoo Search

- 9. Locate the overall best solution in the population.
- 10. Decision point:
 - Yes → Stop the algorithm and move to the final step.
 - No → Loop back to Step 2 and repeat the process.
- 11. Output the best solution found after iterations.

Firefly Algorithm



Firefly Algorithm

- **Step 1: Initialize the first population of fireflies randomly.**
- A group (population) of fireflies is randomly placed within the problem's solution space.
- Each firefly represents a candidate solution.
- The dimensionality of each firefly corresponds to the number of variables in the optimization problem.
- **Step 2: Calculate the fitness of fireflies and assume fitness values as Light Intensity**
- The objective function value (fitness) for each firefly is calculated.
- This fitness value is interpreted as the light intensity emitted by the firefly.
- Brighter fireflies represent better solutions.

Firefly Algorithm

Step 3: Update the firefly position based on the attractiveness

Fireflies move toward **brighter** fireflies.

The **attractiveness** decreases with distance—this simulates how real fireflies see each other.

Movement formula:

$$x_i = x_i + \beta_0 e^{-\gamma r^2} (x_j - x_i) + \alpha \epsilon$$

Where:

- x_i : position of firefly i
- x_j : position of brighter firefly j
- β_0 : attractiveness at distance 0
- γ : light absorption coefficient
- α : randomization parameter
- ϵ : random vector from uniform or Gaussian distribution

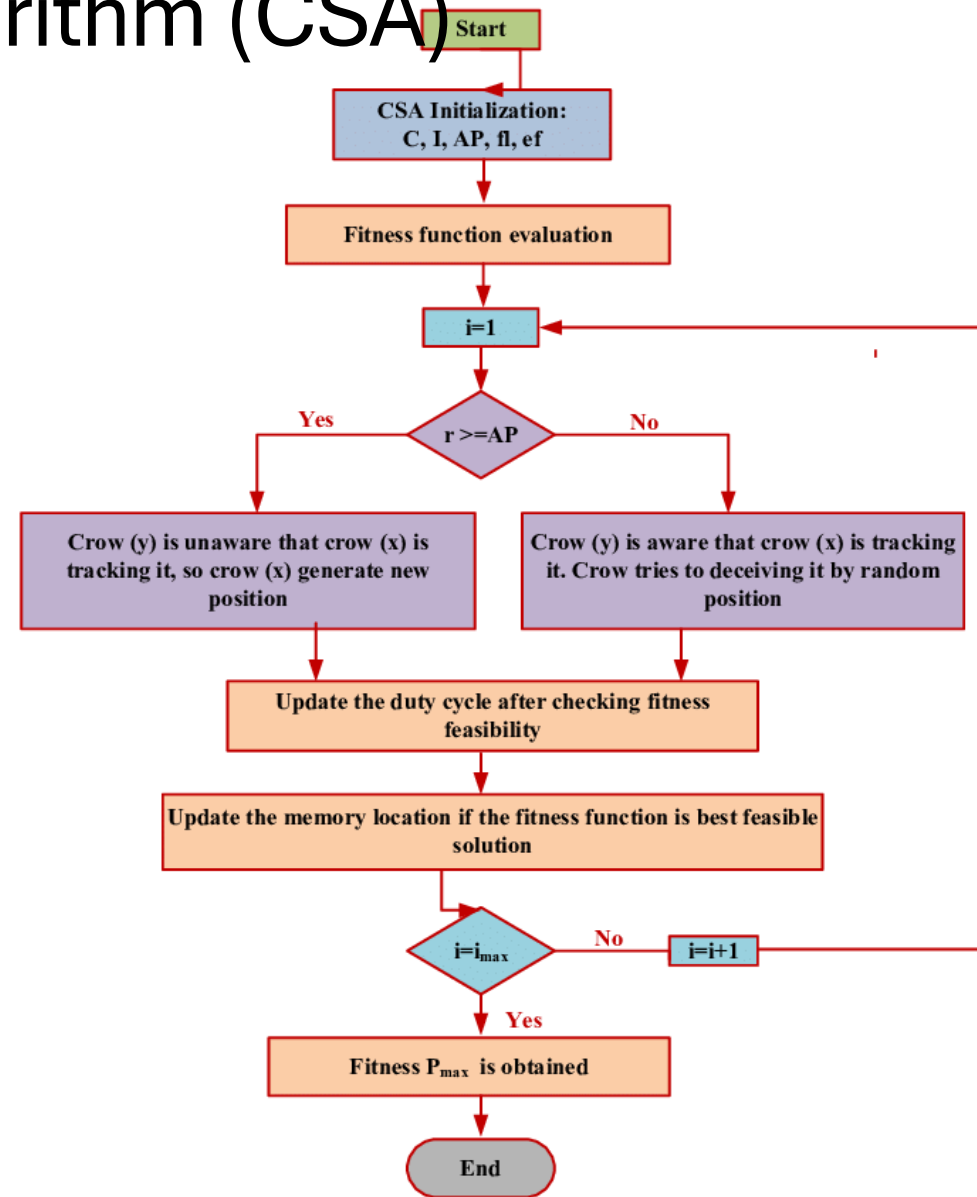
Firefly Algorithm

- **Step 4: Calculate the fitness and update the best firefly position.**
- After moving, each firefly's new position is evaluated.
- The fitness is recalculated at this new position.
- If the new solution is better, it becomes the new position of that firefly.
- Also, the best solution (brightest firefly) is updated if a better solution is found.
- **Step 5: Check termination criteria satisfied?**
- The algorithm checks whether a stopping condition is met.
- Common termination criteria:
 - Maximum number of iterations reached
 - Desired fitness level achieved
 - No significant improvement over a number of iterations

Firefly Algorithm

- **If No: Loop Back**
- If the termination criteria are not satisfied, the algorithm loops back to update firefly positions again and repeats the process.
- **END**
- If the termination criteria are met, the algorithm stops.
- The best solution (position of the brightest firefly) is returned as the output.
- This algorithm mimics natural behavior to balance exploration (via random movement) and exploitation (movement toward better solutions), making it effective for solving complex optimization problems.

Crow Search Algorithm (CSA)



Crow Search Algorithm

- Inspired by the intelligent behavior of crows, especially their food hiding and following behavior.

CSA Initialization: C, I, AP, fl, ef

- **C**: Number of crows (population).
- **I**: Number of iterations (maximum generation).
- **AP**: Awareness probability – the probability that a crow is aware it's being followed.
- **fl**: Flight length – the step size to determine how far a crow can fly in one iteration.
- **ef**: Environment factor or other constraints (if any).
- This step sets up the initial population and parameters of the algorithm.

Fitness Function Evaluation

- The fitness function evaluates how good each crow's current position (solution) is.
- This is similar to evaluating how optimal the crow's food-hiding place is.

Crow Search Algorithm

■ $i = 1$

- The iteration counter i is set to 1.
- The algorithm will iterate through a loop until i reaches the maximum number of iterations i_{max} .

◆ Check Awareness Condition: $r \geq AP$?

- A random number r is generated in the range $[0, 1]$.
- This is compared with AP (Awareness Probability).

► If Yes ($r \geq AP$):

- Crow y is unaware that crow x is following it.
- So, crow x moves to the position of crow y 's memory (best known position).
- This simulates intelligent tracking behavior.

► If No ($r < AP$):

- Crow y is aware of being tracked by crow x .
- To avoid being followed, crow y deceives crow x by moving to a random position.

■ Update the duty cycle after checking fitness feasibility

- After new positions are generated, the algorithm checks whether these new positions are feasible.
- Infeasible positions (like out-of-bound solutions) are corrected.
- Then the system proceeds to update the duty cycle or solution accordingly.

Crow Search Algorithm

Update the memory location if the fitness function is best feasible solution

- Each crow remembers the best position it has found so far.
- If the current position yields a better fitness value, then it updates its memory.

◆ Check if $i = i_{\max}$

- If the current iteration i has reached the maximum allowed iterations ($i_{\max}$), the algorithm proceeds to finish.

► If No:

- Increment the iteration counter $i = i + 1$, and return to the awareness check loop.

► If Yes:

- The algorithm ends and the best solution found ($P_{\max}$) is returned.

■ Fitness $P_{\max}$ is obtained

- The optimal or near-optimal solution is acquired at this point.
- This is the best fitness value achieved throughout all iterations.

Crow Search Algorithm

- Crows follow each other to find better food locations.
- If one crow is being followed and knows it, it misleads others.
- This behavior creates a balance between exploration (random positions) and exploitation (following better positions).
- CSA is used in solving complex optimization problems due to its simplicity and effectiveness.

Hybrid Wolf-Bat Algorithm

- metaheuristic optimization algorithm that combines the characteristics of two different nature-inspired algorithms: the Grey Wolf Optimizer (GWO) and the Bat Algorithm (BA).
- Metaheuristic means finding good solutions to complex problems where traditional methods are inefficient.
- **Grey Wolf Optimizer (GWO):** GWO is inspired by the social hierarchy and hunting behavior of grey wolves.
- It has four main types of wolves: alpha, beta, delta, and omega, representing the leader and followers in the hunting pack.
- mimics the hunting behavior of wolves to search for optimal solutions.

Hybrid Wolf-Bat Algorithm

- **Bat Algorithm (BA):** BA is inspired by the echolocation behavior of bats when searching for prey.
- Bats emit sound waves and listen for echoes to locate food sources.
- The algorithm simulates the movement of bats and their echolocation mechanism to explore the solution space.
- Hybrid Wolf-Bat Algorithm combines the exploration capability of the Bat Algorithm with the social hierarchy and hunting behavior of the Grey Wolf Optimizer to create a more robust optimization algorithm.
- By combining the strengths of both algorithms, HWBA aims to improve convergence speed and solution quality.

Whale Search Algorithm

- The Whale Optimization Algorithm (WOA) is a relatively recent nature-inspired algorithm that mimics the social behavior of humpback (large-winged) whales.
- It was proposed by Seyedali Mirjalili, in 2016.
- The WOA is inspired by the hunting behavior of humpback whales, where they cooperate to encircle their prey.
- Here's a simplified explanation of how the Whale Optimization Algorithm works:
- Initialization: Start with a population of potential solutions (whales) randomly distributed in the search space.
- Encircling Prey: Each whale adjusts its position towards the best solution (prey) found so far in the search space. This movement is similar to the whales encircling their prey cooperatively.

Whale Search Algorithm

- Search for Prey: Whales that haven't found the prey yet explore the search space randomly, simulating their movement in the ocean. This phase introduces diversity and helps avoid getting stuck in local optima.
- Update: Update the best solution found so far and adjust parameters like position, velocity, or fitness of the whales.
- Termination: Repeat steps 2-4 until a termination criterion is met (e.g., a maximum number of iterations reached, or a satisfactory solution is found).
- The key components of the Whale Optimization Algorithm include the exploration and exploitation phases.
- During exploration, the algorithm randomly explores the search space to avoid getting trapped in local optima (non-optimal solutions).
- Meanwhile, during exploitation, the algorithm converges towards promising regions of the search space where good solutions are likely to be found.

Grasshopper Optimization

- Grasshopper Optimization Algorithm (GOA) is an optimization algorithm inspired by the swarming behavior of grasshoppers.
- It mimics the collective intelligence and foraging (searching & collecting of food) behavior of grasshopper populations to solve optimization problems.
- The algorithm is based on the following key components:
- Grasshopper Movement: Grasshoppers adjust their positions iteratively based on two main factors: attraction toward the best solution found so far and attraction/repulsion from other grasshoppers.
- Attractiveness: The attractiveness of solutions is determined by their fitness values. Better solutions attract more grasshoppers.

Grasshopper Optimization

- Leapfrogging (quickly adopt): Grasshoppers leap toward better solutions while avoiding worse ones, similar to the behavior of grasshoppers jumping from one place to another.
- Local Search: Local search operators can be integrated into the algorithm to further improve solution quality.
- GOA has been applied to various optimization problems, including engineering design, data mining, and machine learning.
- Its simplicity and efficiency make it a popular choice for solving complex optimization tasks.